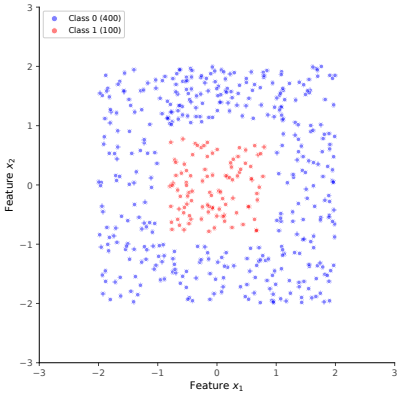
A toy learning problem: a central square



Input: two real-valued features, $x = (x_1, x_2) \in \mathbb{R}^2$.

Target: class label $y \in \{0, 1\}$.

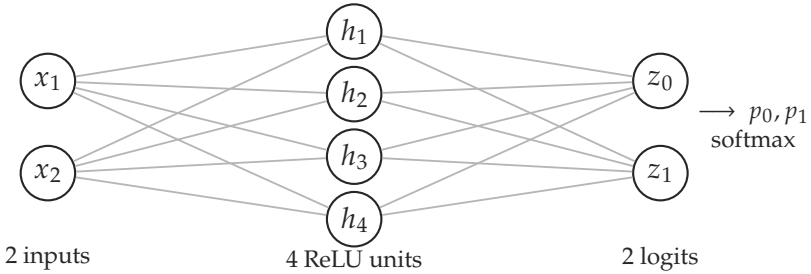
Blue (class 0): 400 points in outer rectangles.

Red (class 1): 100 points in the central square.

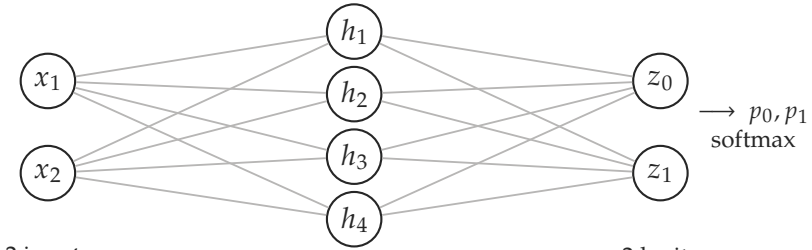
Question: can a model learn a rule that isolates the middle?

Points are sampled uniformly within each rectangle. One straight-line boundary cannot separate the two classes.

A small network to capture a non-linear boundary



A small network to capture a non-linear boundary



$$\underbrace{a = xW_1 + b_1}_{\text{weighted sums}} \longrightarrow \underbrace{h = \max(0, a)}_{\text{ReLU, componentwise}}$$

$$\underbrace{z = hW_2 + b_2}_{\text{unbounded scores (logits)}} \longrightarrow \underbrace{p = \text{softmax}(z)}_{\text{two class probabilities}}$$

What, exactly, is computed?

One observation is a row vector $x \in \mathbb{R}^{1\times 2}$.

For hidden neuron $j = 1, \ldots, 4$:

$$a_j = \sum_{i=1}^{2} x_i W_{1,ij} + b_{1,j}, \qquad h_j = \underbrace{\max(0, a_j)}_{\text{ReLU}}.$$

For output class $k = 0, 1$:

$$z_k = \sum_{j=1}^{4} h_j W_{2,jk} + b_{2,k}.$$

Logits $z_k \in \mathbb{R}$ are scores, *not* probabilities.

$$p_k = \frac{e^{z_k}}{e^{z_0} + e^{z_1}}, \qquad p_0 + p_1 = 1.$$

Dimensions and parameters

Object	Shape	Scalars
x	1×2	input
W_1	2×4	8
b_1	1×4	4
a, h	1×4	intermediate
W_2	4×2	8
b_2	1×2	2
z, p	1×2	output

Total trainable parameters:

$$\boxed{8 + 4 + 8 + 2 = 22}.$$

Weights scale signals; biases shift them. Both are learned.

Initialization W_1, b_1, W_2, b_2 – why does it matter?

1. Break the symmetry

If two hidden neurons start with identical incoming weights and biases, they produce identical outputs for every input. Their gradients are identical under symmetric training; they remain copies, wasting capacity.

Random independent weights make neurons different. Biases can initially be zero.

2. Control the size of signals

For a weighted sum $a_j = \sum_{i=1}^{n_{\text{in}}} w_{ij} x_i$ with independent, zero-mean w and x:

$$\text{Var}(a_j) \approx n_{\text{in}} \, \text{Var}(w) \, \text{Var}(x).$$

Too-small weights: weak signals.
Too-large weights: amplified signals and potentially unstable gradients.

Note: ReLU sets negative sums to zero, changing activation variance.

The variance relation is a heuristic under simplifying independence assumptions, not an exact guarantee for a trained network.

Glorot/Xavier vs. He/Kaiming initialization

Method	Sampling distribution	Motivation / common use
Glorot / Xavier normal	$w \sim \mathcal{N}\left(0, \frac{2}{n_{\text{in}} + n_{\text{out}}}\right)$	Balance forward and backward variance (tanh, linear).
Glorot / Xavier uniform	$w \sim U[-a, a]$ (uniform density)	Same variance as Xavier normal, but bounded samples.
He / Kaiming normal	$w \sim \mathcal{N}\left(0, \frac{2}{n_{\text{in}}}\right)$	Accounts approximately for ReLU zeroing negative activations.

Fan-in n_{in} counts inputs to a neuron; fan-out n_{out} counts its outgoing connections. For uniform Xavier, $a = \sqrt{6/(n_{\text{in}} + n_{\text{out}})}$; for $\mathcal{N}(0, v)$, v is the *variance*.

Our network: ReLU: He normal ($\sigma = 1$); logits: Xavier normal ($\sigma = \sqrt{1/3} \approx 0.577$). Biases: 0.

Sources: Glorot & Bengio (2010); He et al. (2015).

Actual random initialization

We draw the weights with seed 11, round each weight to 0.01, then use these rounded weights for all plots and calculations. Biases start at zero.

Input → hidden layer

$$W_1 = \begin{bmatrix} 0.03 & 1.36 & 1.22 & -0.51 \\ -0.30 & -0.53 & 0.57 & -0.06 \end{bmatrix}$$

$$b_1 = \begin{bmatrix} 0 & 0 & 0 & 0 \end{bmatrix}$$

Shape 2×4: each column feeds one hidden neuron.

Hidden → output layer

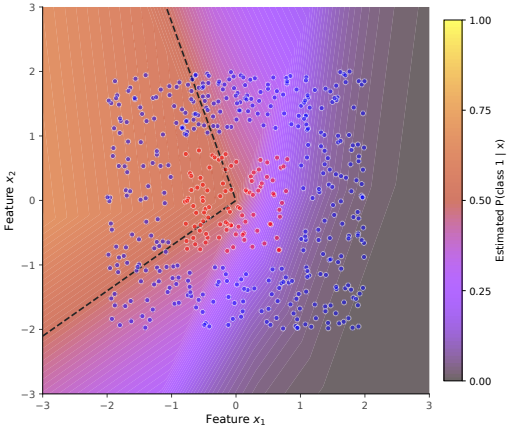
$$W_2 = \begin{bmatrix} 0.43 & -1.07 \\ 0.90 & -0.06 \\ 0.39 & -0.08 \\ -0.22 & 0.27 \end{bmatrix}$$

$$b_2 = \begin{bmatrix} 0 & 0 \end{bmatrix}$$

Shape 4×2: each column feeds one output logit.

Example observation for analysis: $x = (0.33, -0.47)$, target class $y = 1$ (red point).

Before training: the output map

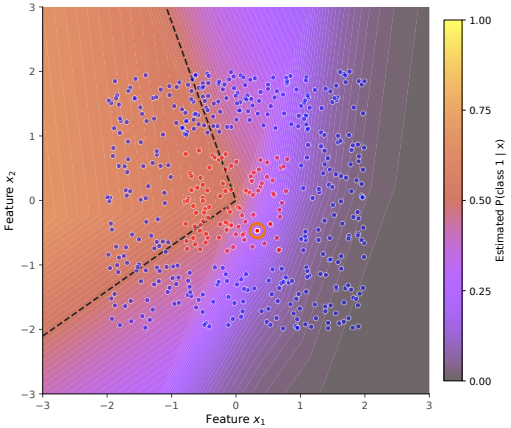


Each pixel is a different input (x_1, x_2). Color encodes the **initial** softmax probability $p_1(x)$.

The **dashed curve** is the decision boundary $p_1 = 0.5$ (equal class probabilities).

No optimization yet: map entirely from random weights.

Before training: the output map



Each pixel is a different input (x_1, x_2). Color encodes the **initial** softmax probability $p_1(x)$.

The **dashed curve** is the decision boundary $p_1 = 0.5$ (equal class probabilities).

No optimization yet: map entirely from random weights.

Orange ring: our red class-1 point $x = (0.33, -0.47)$.

The network currently predicts **class 0**: $p_0 \approx 0.7231$ and $p_1 \approx 0.2769$.

First forward pass: weighted sums and ReLU

For the highlighted input $x = (0.33, -0.47)$, each hidden 'preactivation' is $a_j = x_1 W_{1,1j} + x_2 W_{1,2j} + b_{1,j}$, with $b_1 = 0$.

$$a_1 = (0.33)(0.03) + (-0.47)(-0.30) = \boxed{0.1509}$$

$$a_2 = (0.33)(1.36) + (-0.47)(-0.53) = \boxed{0.6979}$$

$$a_3 = (0.33)(1.22) + (-0.47)(0.57) = \boxed{0.1347}$$

$$a_4 = (0.33)(-0.51) + (-0.47)(-0.06) = \boxed{-0.1401}$$

Apply the **rectified linear unit** $\mathrm{ReLU}(a_j) = \max(0, a_j)$ to each number:

$$\boxed{h = (0.1509,\ 0.6979,\ 0.1347,\ 0.0000)}.$$

ReLU leaves positive values unchanged and replaces negative values by 0. Here hidden neuron 4 is inactive for this input.

Then, for the same input: logits and softmax

The hidden activations are $h = (0.1509, 0.6979, 0.1347, 0)$.
Multiply by W_2 and add $b_2 = 0$:

$$z_0 = (0.1509)(0.43) + (0.6979)(0.90) + (0.1347)(0.39) + (0)(-0.22) = \boxed{0.7455}$$

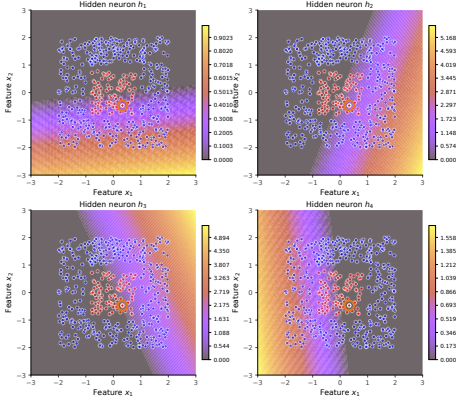
$$z_1 = (0.1509)(-1.07) + (0.6979)(-0.06) + (0.1347)(-0.08) + (0)(0.27) = \boxed{-0.2141}.$$

Logits $z = (0.7455, -0.2141)$ are unbounded scores.
Softmax turns them into competing probabilities:

$$p_k = \frac{e^{z_k}}{e^{z_0} + e^{z_1}}, \qquad p_0 = \frac{e^{0.7455}}{e^{0.7455} + e^{-0.2141}} = \boxed{0.7231}, \quad p_1 = \boxed{0.2769}.$$

Prediction: $\hat{y} = \arg\max_k p_k = \mathbf{0}$ . . . but the target is $y = \mathbf{1}$.

First forward pass: four hidden neurons feature maps



For every pixel x, plot $h_j(x) = \max(0, xW_{1,:j} + b_{1,j})$.

Each unit is **zero on one side** of its line $a_j(x) = 0$, and increases linearly on the other side.

The class probabilities depend on a *weighted combination* of these four feature maps.

$h = (0.1509,\ 0.6979,\ 0.1347,\ 0.0000)$

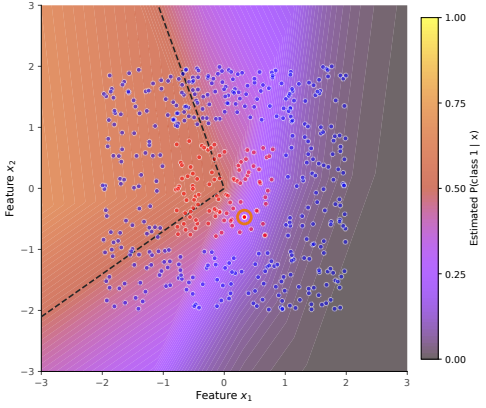
One observation, from input to decision, summary

Stage	Numerical result for the selected class-1 point
Input and target	$x = (0.33, -0.47), \quad y = 1$
Preactivations	$a = (0.1509, 0.6979, 0.1347, -0.1401)$
ReLU activations	$h = (0.1509, 0.6979, 0.1347, 0.0000)$
Output logits	$z = (0.7455, -0.2141)$
Softmax output	$p = (0.7231, 0.2769)$, with $p_0 + p_1 = 1$
Predicted label	$\hat{y} = 0$ (because $p_0 > p_1$)

What we have: a fully specified deterministic forward computation.

What we do not yet have: a measure of the error, a gradient, or a rule for changing the 22 parameters. That is the next step.

A wrong answer from the network: but how wrong?



Again: our selected red point.

$x = (0.33, -0.47)$, true class $y = 1$.

The initialized network gives

$$z = (0.7455, -0.2141),$$

$$p = (0.7231, 0.2769).$$

It predicts class 0, because $p_0 > p_1$.

Question: how can we assign a numerical penalty to this prediction?

We need a loss function ℓ measuring the mistake on this one observation.

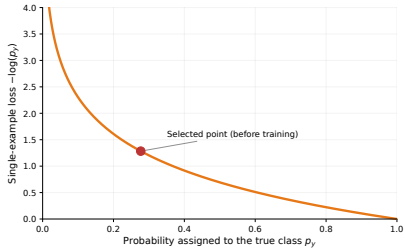
Cross-entropy: a loss for class probabilities

For K **mutually exclusive classes**, let p_k be the predicted probability and $t_k \in \{0, 1\}$ the target indicator (**one-hot target**): $\sum_k t_k = 1$.

Categorical cross-entropy for one example:

$$\boxed{\ell(t, p) = -\sum_{k=0}^{K-1} t_k \log p_k}$$

$$\ell = -\log p_y \quad \text{(one-hot target)}.$$



Here $t = (0, 1)$, so only the true class contributes:

$$\ell = -\log(0.2769) = \boxed{1.2839}.$$

Natural log; $0 < p_y \leq 1$. A 'confidently correct' prediction has loss near 0; as $p_y \to 0$, the loss grows without bound.

Gradient: how should the 22 parameters change?

We have 22 adjustable parameters $\theta = (\theta_1, \ldots, \theta_{22})$. How should we change them to reduce the loss $\ell(\theta; x, y)$ for our fixed training example (x, y)?

1. Consider one parameter at a time. The partial derivative measures how the loss changes when we change just parameter θ_j, keeping all others fixed:

$$\frac{\partial \ell}{\partial \theta_j} = \lim_{\varepsilon \to 0} \frac{\ell(\theta + \varepsilon e_j) - \ell(\theta)}{\varepsilon}.$$

Here e_j is a vector with 1 at position j and 0 elsewhere.

- Positive derivative: increasing θ_j increases loss $\Rightarrow$ we should decrease θ_j.
- Negative derivative: increasing θ_j decreases loss $\Rightarrow$ we should increase θ_j.
- Magnitude: how sensitive the loss is to that parameter.

2. Collect all derivatives into the gradient:

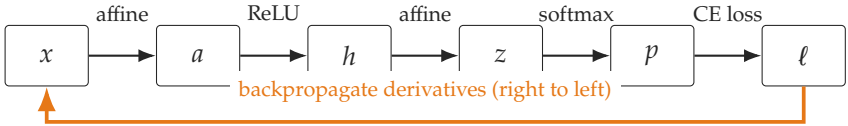
$$g = \nabla_\theta \ell = \left(\frac{\partial \ell}{\partial \theta_1}, \ldots, \frac{\partial \ell}{\partial \theta_{22}} \right).$$

3. Change all parameters simultaneously:

$$\boxed{\theta^+ = \theta - \eta g}, \qquad \eta > 0 \quad \text{(learning rate)}.$$

Why the minus sign? For a sufficiently small step, $\ell(\theta - \eta g) \approx \ell(\theta) - \eta \|g\|^2$.

Backpropagation: apply the chain rule backward



Forward pass: store a, h, z, p for the chosen input.

Backward pass: compute the derivative of the loss with respect to each intermediate quantity, then each parameter.

Reuse: a once-computed derivative at an intermediate node is shared (used) by all upstream paths.

Chain rule (scalar example):

$$\frac{d\ell}{dw} = \frac{d\ell}{dz}\frac{dz}{dw}.$$

Our convenient intermediate gradients:

$$\delta_z := \frac{\partial \ell}{\partial z}, \qquad \delta_a := \frac{\partial \ell}{\partial a}.$$

Here δ_z has 2 entries, δ_a has 4 entries.

Softmax + cross-entropy: a nice simplification

For *any number of classes* K, $\quad p_k = \frac{e^{z_k}}{\sum_{j=0}^{K-1} e^{z_j}}, \qquad \ell = -\sum_{k=0}^{K-1} t_k \log p_k, \qquad \sum_k t_k = 1.$

Since $\log p_k = z_k - \log\left(\sum_j e^{z_j}\right)$,

$$\ell = -\left(\sum_k t_k z_k - \underbrace{\sum_k t_k}_{=1} \log\left(\sum_j e^{z_j}\right)\right) = -\sum_k t_k z_k + \log\left(\sum_j e^{z_j}\right).$$

Differentiating with respect to output logit z_m gives

$$\frac{\partial \ell}{\partial z_m} = \underbrace{\frac{\partial}{\partial z_m}\left(-\sum_k t_k z_k\right)}_{-t_m} + \underbrace{\frac{\partial}{\partial z_m} \log\left(\sum_j e^{z_j}\right)}_{\frac{e^{z_m}}{\sum_j e^{z_j}} = p_m} = p_m - t_m.$$

Again: with $p = (0.7231, 0.2769)$ and $t = (0, 1)$, $\boxed{\delta_z = p - t = \begin{bmatrix} 0.7231 & -0.7231 \end{bmatrix}}$.

Numerical backpropagation, part 1: output weights

For our point, $h = (0.1509, 0.6979, 0.1347, 0)$ and $\delta_z = \begin{bmatrix} 0.7231 & -0.7231 \end{bmatrix}$.

Since the logits are $z = hW_2 + b_2$, $\frac{\partial \ell}{\partial W_{2,jk}} = \frac{\partial \ell}{\partial z_k} \frac{\partial z_k}{\partial W_{2,jk}} = \delta_{z,k} h_j$

$$\boxed{\frac{\partial \ell}{\partial W_2} = h^T \delta_z}, \qquad \boxed{\frac{\partial \ell}{\partial b_2} = \delta_z}.$$

All 8 weight gradients:

$$\frac{\partial \ell}{\partial W_2} = \begin{bmatrix} 0.1091 & -0.1091 \\ 0.5046 & -0.5046 \\ 0.0974 & -0.0974 \\ 0.0000 & 0.0000 \end{bmatrix}$$

Both bias gradients:

$$\frac{\partial \ell}{\partial b_2} = \begin{bmatrix} 0.7231 & -0.7231 \end{bmatrix}.$$

Example: $\partial \ell / \partial W_{2,2,0} = h_2 \delta_{z,0}$

$= (0.6979)(0.7231) = \boxed{0.5046}$.

Numerical backpropagation: reaching the hidden neurons

We already know how the loss depends on the two output logits: $\delta_z = \begin{bmatrix} 0.7231 & -0.7231 \end{bmatrix}$.

Step 1: Each hidden neuron connects to both outputs.
For hidden neuron j, the gradient signals from both outputs are added, each multiplied by the corresponding connection weight:

$$\frac{\partial \ell}{\partial h_j} = \underbrace{\frac{\partial \ell}{\partial z_0}\frac{\partial z_0}{\partial h_j}}_{\text{path through output 0}} + \underbrace{\frac{\partial \ell}{\partial z_1}\frac{\partial z_1}{\partial h_j}}_{\text{path through output 1}} = \delta_{z,0} W_{2,j0} + \delta_{z,1} W_{2,j1}.$$

Step 2: ReLU either passes or blocks this signal.
Recall $h_j = \mathrm{ReLU}(a_j) = \max(0, a_j)$. Its derivative is 1 when $a_j > 0$ and 0 when $a_j < 0$.

$$\boxed{\delta_{a,j} = \frac{\partial \ell}{\partial a_j} = (\delta_{z,0} W_{2,j0} + \delta_{z,1} W_{2,j1}) \cdot \mathrm{ReLU}'(a_j)}$$

Example: first hidden neuron ($a_1 = 0.1509 > 0$): $\delta_{a,1} = (0.7231 \cdot 0.43 - 0.7231 \cdot (-1.07)) \cdot 1 \approx 1.0846$.
The fourth neuron has $a_4 < 0$, so its gradient is zero.

$$\boxed{\delta_a = \begin{bmatrix} 1.0846 & 0.6941 & 0.3398 & 0.0000 \end{bmatrix}}$$

At $a_j = 0$, ReLU is not differentiable; we conventionally use derivative 0.

Numerical backpropagation: hidden weights and biases

We now know how the loss depends on each hidden neuron's pre-activation a_j.
Step 3: Find the gradients of the incoming weights.
Recall how hidden neuron j computes its pre-activation:

$$a_j = x_1 W_{1,1j} + x_2 W_{1,2j} + b_{1,j}.$$

Changing an incoming weight changes a_j proportionally to the corresponding input x_i. By the chain rule:

$$\boxed{\frac{\partial \ell}{\partial W_{1,ij}} = x_i \delta_{a,j}, \qquad \frac{\partial \ell}{\partial b_{1,j}} = \delta_{a,j}.}$$

Example: first hidden neuron, with $x = (0.33, -0.47)$:

$$\frac{\partial \ell}{\partial W_{1,1,1}} = 0.33 \cdot 1.0846 = \quad 0.3579,$$
$$\frac{\partial \ell}{\partial W_{1,2,1}} = -0.47 \cdot 1.0846 = -0.5098.$$

All hidden-layer gradients:

$$\frac{\partial \ell}{\partial W_1} = x^T \delta_a = \begin{bmatrix} 0.3579 & 0.2291 & 0.1121 & 0.0000 \\ -0.5098 & -0.3262 & -0.1597 & 0.0000 \end{bmatrix} \qquad \frac{\partial \ell}{\partial b_1} = \begin{bmatrix} 1.0846 & 0.6941 & 0.3398 & 0.0000 \end{bmatrix}$$

Interpretation: All incoming weights of an inactive ReLU neuron receive zero gradient for this example.

Update the parameters: one actual gradient step

(deliberately visible learning rate $\eta = 0.20$ for this single class-1 example)

$$\boxed{\theta^{+} = \theta - \eta \nabla_{\theta} \ell}, \qquad \theta = (W_1, b_1, W_2, b_2).$$

Updated output weights (4×2)

$$W_2^{+} = \begin{bmatrix} 0.4082 & -1.0482 \\ 0.7991 & 0.0409 \\ 0.3705 & -0.0605 \\ -0.2200 & 0.2700 \end{bmatrix}$$

Updated output biases (1×2)

$$b_2^{+} = \begin{bmatrix} -0.1446 & 0.1446 \end{bmatrix}$$

Updated hidden weights (2×4)

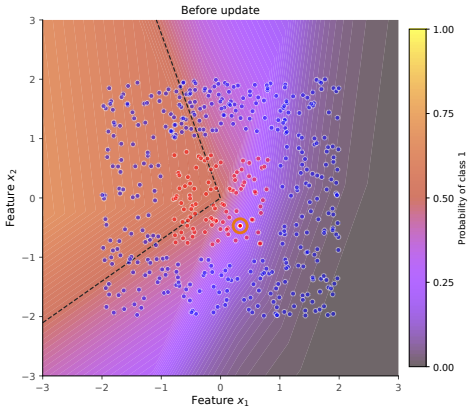
$$W_1^{+} = \begin{bmatrix} -0.0416 & 1.3142 & 1.1976 & -0.5100 \\ -0.1980 & -0.4648 & 0.6019 & -0.0600 \end{bmatrix}$$

Updated hidden biases (1×4)

$$b_1^{+} = \begin{bmatrix} -0.2169 & -0.1388 & -0.0680 & 0.0000 \end{bmatrix}$$

Notice the unchanged fourth column of W_1: its ReLU was inactive.

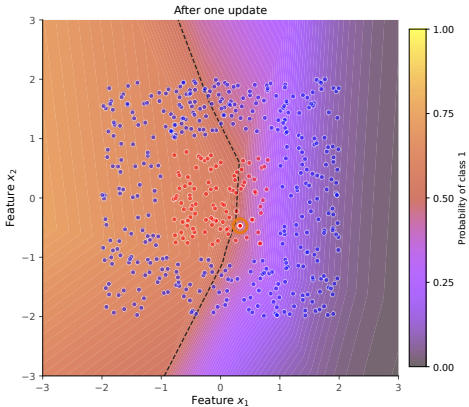
Refrain: the same point after one update



Same dataset, same point, same color scale (p_1), same architecture. Only weights and biases change.

	Before	After
$p_1(x)$	0.2769	0.4703
$\ell(x)$	1.2839	0.7544
Class	0	0

Refrain: the same point after one update

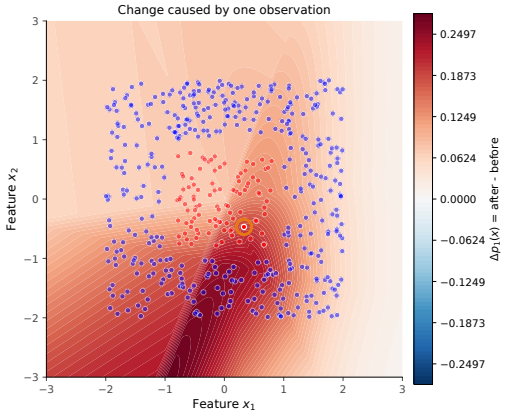


Same dataset, same point, same color scale (p_1), same architecture. Only weights and biases change.

	Before	After
$p_1(x)$	0.2769	0.4703
$\ell(x)$	1.2839	0.7544
Class	0	0

The probability of the true class increases, and the loss falls—even though the predicted class remains wrong.

One point changed a function on the whole plane



The color now displays a **difference**, not a probability:

$$\Delta p_1(x) = p_1(x; \theta^+) - p_1(x; \theta).$$

Red: p_1 rose after the update; **blue:** it fell.

The orange ring marks the *only* observation whose gradient was used.

Why does the whole picture change? The same 22 parameters determine predictions for every x.

Improving one example may harm another.

From a chosen observation to stochastic gradient descent

The previous update used **one** chosen example. In **stochastic gradient descent (SGD)**, we sample one observation at each step and update using its gradient:

$$\boxed{\theta_{t+1} = \theta_t - \eta \nabla_\theta \ell(\theta_t; x_{i_t}, y_{i_t})}, \qquad i_t \text{ sampled at step } t.$$

What varies from step to step?

1. The sampled observation (x_{i_t}, y_{i_t}).
2. Its forward pass and error signal.
3. The 22 gradients and updated parameters.

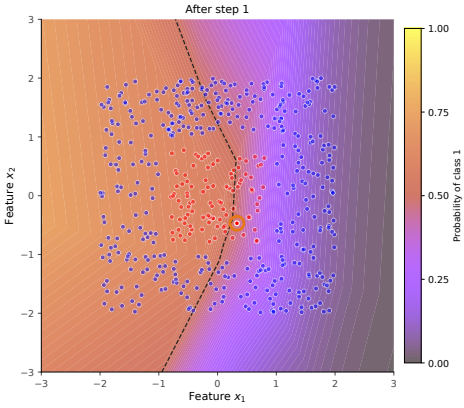
What stays the same? The network, loss formula, chain rule, and update rule.

Our continuation (seed 18):

Step	Index	$\approx (x_1, x_2)$	y
1	400	$(0.33, -0.47)$	1
2	199	$(-0.74, -1.25)$	0
3	445	$(-0.14, -0.78)$	1

Step 1 was chosen; steps 2–3 were sampled uniformly without replacement (fixed seed 18).

Refrain: single-example SGD changes the map again

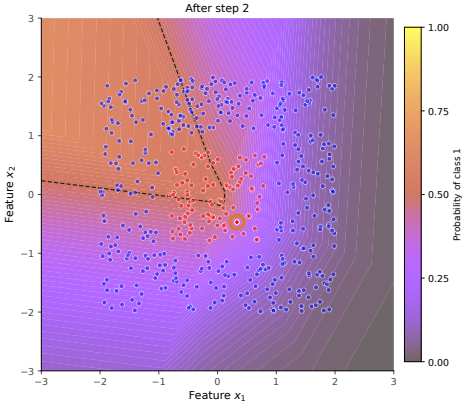


Each image is the **entire** probability map after an additional update. The orange ring remains on the original point ($y = 1$).

Update	Training label	p_1 at ring
0 (initial)	—	0.2769
1	1	0.4703
2	0	0.3763
3	1	0.4949

Step 1: loss of the original example drops to 0.7544.

Refrain: single-example SGD changes the map again

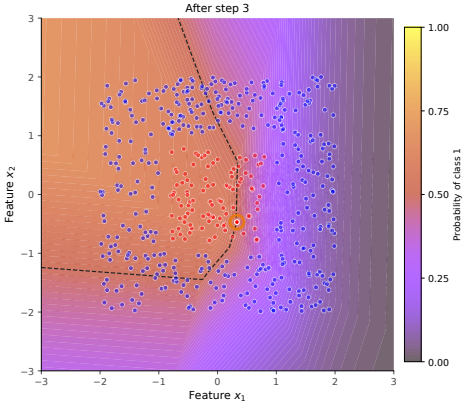


Each image is the **entire** probability map after an additional update. The orange ring remains on the original point ($y = 1$).

Update	Training label	p_1 at ring
0 (initial)	—	0.2769
1	1	0.4703
2	0	0.3763
3	1	0.4949

Step 2: class-0 sample $\approx (-0.74, -1.25)$: loss $0.9271 \rightarrow 0.3892$. At the ring, p_1 falls.

Refrain: single-example SGD changes the map again



Each image is the **entire** probability map after an additional update. The orange ring remains on the original point ($y = 1$).

Update	Training label	p_1 at ring
0 (initial)	—	0.2769
1	1	0.4703
2	0	0.3763
3	1	0.4949

Step 3: class-1 sample $\approx (-0.14, -0.78)$: loss $0.9381 \rightarrow 0.5685$. At the ring, p_1 rises.

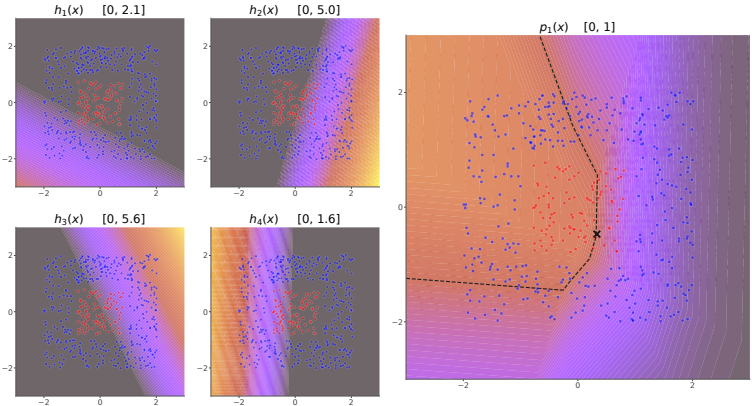
What these three updates establish—and what they do not

The mechanism is complete: one observation → forward pass → single-example cross-entropy → chain rule → 22 partial derivatives → one parameter update. Repeat with another sampled observation.

We have demonstrated	We have *not* yet demonstrated
Numerical gradients and updates	Training on all examples / average loss
A local loss decrease for the updated example	Monotone improvement on other examples
Changing output maps over three steps	Minibatches, learning-rate schedules, convergence…

Question for the next segment: why might a long sequence of such noisy, single-example updates improve the classifier overall?

SGD refrain: five views of the same network



After SGD step 3

Starting point for this sequence

p_1 at original red point:

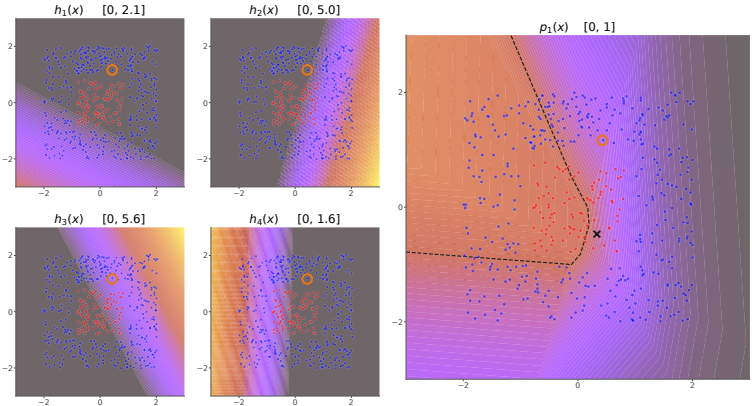
0.4949

No new observation yet

Next overlays: steps 4-7

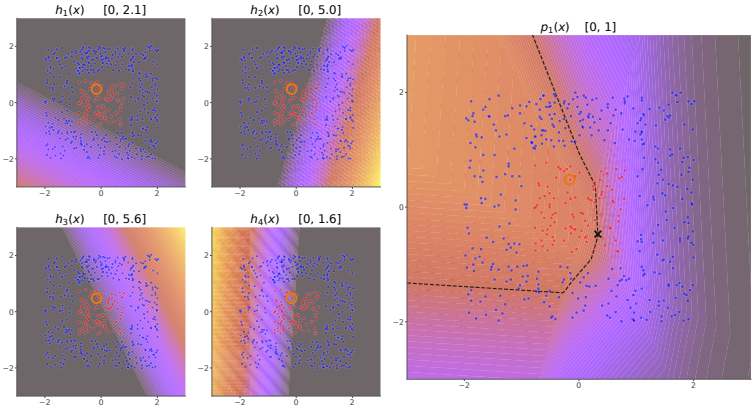
Dots: blue = class 0, red = class 1; orange ring = current training example; black × = original red point. Each hidden neuron has its own fixed scale across overlays; p_1 stays in [0,1]. Dashed contour: $p_1 = 0.5$.

SGD refrain: five views of the same network



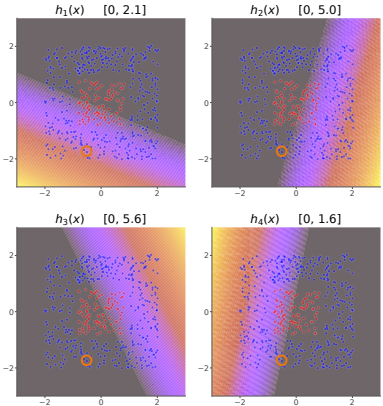
Dots: blue = class 0, red = class 1; orange ring = current training example; black × = original red point. Each hidden neuron has its own fixed scale across overlays; p_1 stays in [0,1]. Dashed contour: $p_1 = 0.5$.

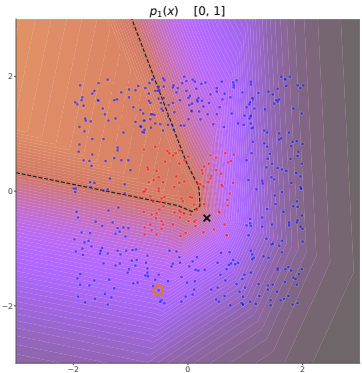
SGD refrain: five views of the same network



Dots: blue = class 0, red = class 1; orange ring = current training example; black × = original red point. Each hidden neuron has its own fixed scale across overlays; p_1 stays in [0,1]. Dashed contour: $p_1 = 0.5$.

SGD refrain: five views of the same network





After SGD step 6

Sample #109, class 0

$x = (-0.52, -1.73)$

One-point loss: 0.644 → 0.264

p_1 at original point:

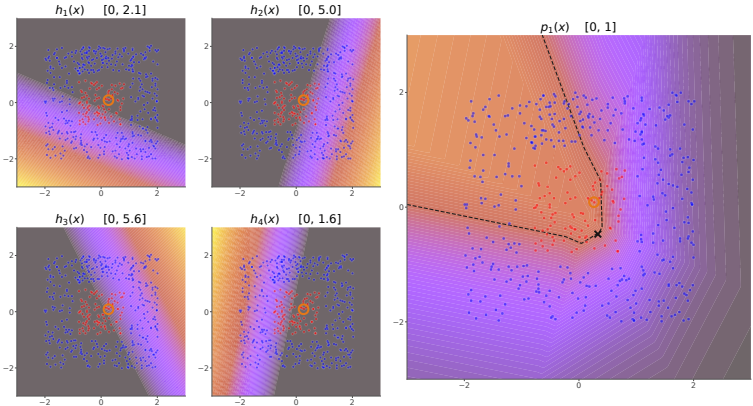
0.499 → 0.424

Changed hidden maps: h_1, h_4

Unchanged: h_2, h_3

Dots: blue = class 0, red = class 1; orange ring = current training example; black × = original red point. Each hidden neuron has its own fixed scale across overlays; p_1 stays in [0,1]. Dashed contour: $p_1 = 0.5$.

SGD refrain: five views of the same network



After SGD step 7

Sample #441, class 1

$x = (+0.26, +0.09)$

One-point loss: 0.756 → 0.595

p_1 at original point:

0.424 → 0.497

Changed hidden maps: h_2, h_3

Unchanged: h_1, h_4

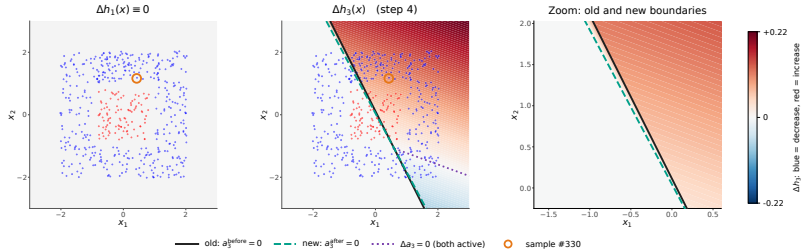
Dots: blue = class 0, red = class 1; orange ring = current training example; black × = original red point. Each hidden neuron has its own fixed scale across overlays; p_1 stays in [0,1]. Dashed contour: $p_1 = 0.5$.

Why do some hidden-layer maps stay unchanged?

For hidden neuron j, a single-example backpropagation update is

$$\delta_{a,j} = \left[(p - t)W_2^T\right]_j \mathbf{1}_{a_j > 0}, \qquad \frac{\partial \ell}{\partial W_{1,:,j}} = x^T \delta_{a,j}, \qquad \frac{\partial \ell}{\partial b_{1,j}} = \delta_{a,j}.$$

For sample #330 (class 0), $a_1, a_2, a_4 \leq 0$: only h_3 changes its incoming parameters. The other three hidden maps stay identical.



Difference maps: $\Delta h_j(x) = h_j(x; \theta_4) - h_j(x; \theta_3)$ everywhere on the plane. Red = increase; blue = decrease; white = no change.

What are we minimizing? From one observation to 500

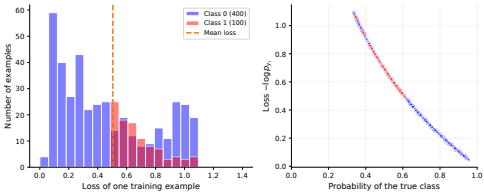
For input x_i, true class $y_i \in \{0, 1\}$ and predicted class probabilities $p(x_i; \theta)$, the **cross-entropy loss** for one observation is

$$\ell_i(\theta) = -\log p_{y_i}(x_i; \theta).$$

The network parameters $\theta \in \mathbb{R}^{22}$ contain all weights and biases. Training on the whole dataset means minimizing the **empirical risk**

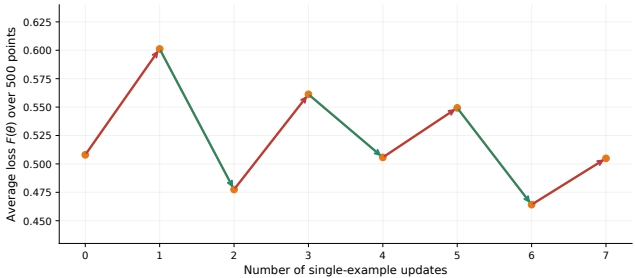
$$F(\theta) = \frac{1}{N} \sum_{i=1}^{N} \ell_i(\theta), \qquad N = 500.$$

Important distinction: ℓ_i depends on a particular example; F is the average over all training examples. They are different functions of the same 22 parameters. The objective is F, not any selected ℓ_i. Here: 400 examples of class 0 (blue), 100 of class 1 (red).



Did 7 successful single-point steps reduce the total loss?

All seven earlier updates reduced the loss on *their sampled point*. But the training objective F does not decrease after every such update.



At step 7:
$F(\theta_6) = 0.4642 \longrightarrow F(\theta_7) = 0.5049$. The sampled example's loss nevertheless decreased.

Every parameter update changes the network's predictions *at other points too*. Why should optimizing randomly selected individual examples help minimize the average loss over the entire dataset?

Three meanings of a training step

For any estimated gradient g_t, the update rule has the same form:

$$\theta_{t+1} = \theta_t - \eta g_t, \qquad \eta > 0 \text{ (learning rate).}$$

At current θ_t, full-batch computes ∇F exactly; sampled methods estimate it. I_t is a random index and $\mathcal{B}_t$ a minibatch. "SGD" usually includes minibatch SGD.

Method	**Gradient used at step t**	**Examples per step**
Full-batch GD	$g_t = \frac{1}{500} \sum_{i=1}^{500} \nabla \ell_i(\theta_t)$	500
Minibatch SGD	$g_t = \frac{1}{B} \sum_{i \in \mathcal{B}_t} \nabla \ell_i(\theta_t)$	B
Single-example SGD	$g_t = \nabla \ell_{I_t}(\theta_t)$	1

What is an iteration, a minibatch, and an epoch?

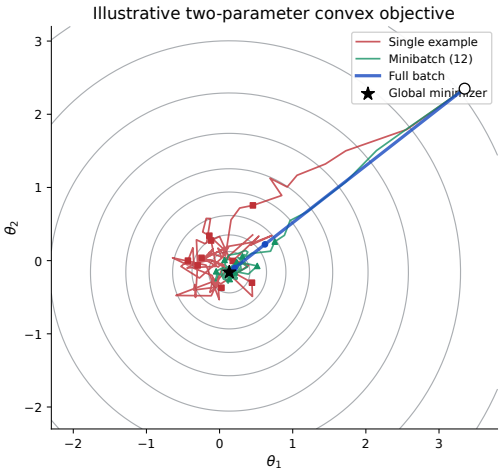
Iteration/step: one parameter update.
Minibatch: examples used to calculate this update.
Epoch: one complete pass through the 500 training examples (assuming shuffled sampling *without replacement*).

	B	Updates / epoch	Examples processed / epoch
Single-example SGD	1	500	500
Minibatch SGD	20	25	500
Full-batch GD	500	1	500

One *epoch* does not mean the same number of parameter updates. A fair comparison needs a declared budget (epochs, processed examples, updates, or actual runtime); these are not interchangeable. Vectorized minibatches can be much faster on modern hardware.

Experiment below: same initialization θ_7, learning rate $\eta = 0.20$, shuffled data (fixed seed), 30 epochs, no regularization. Only batch size differs.

Three optimization trajectories: an interpretable example



Not a projection of our network. A separate convex toy problem:

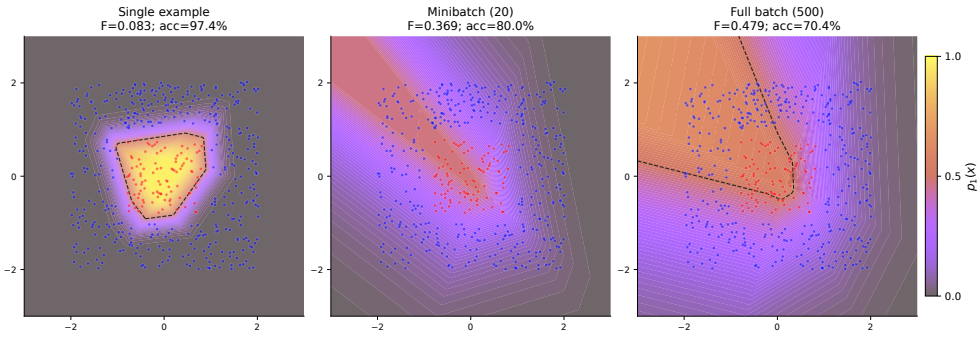
$$f_i(\theta) = \tfrac{1}{2}\|\theta - a_i\|^2,$$
$$F(\theta) = \tfrac{1}{N}\sum_i f_i(\theta).$$

Here $\theta = (\theta_1, \theta_2)$, and the contour lines are equal values of F.

Blue: full gradient, green: $B = 12$, red: $B = 1$. Each colored trajectory shows **75 updates**, not an equal computational budget.

Smaller batches produce noisier directions near the optimum.

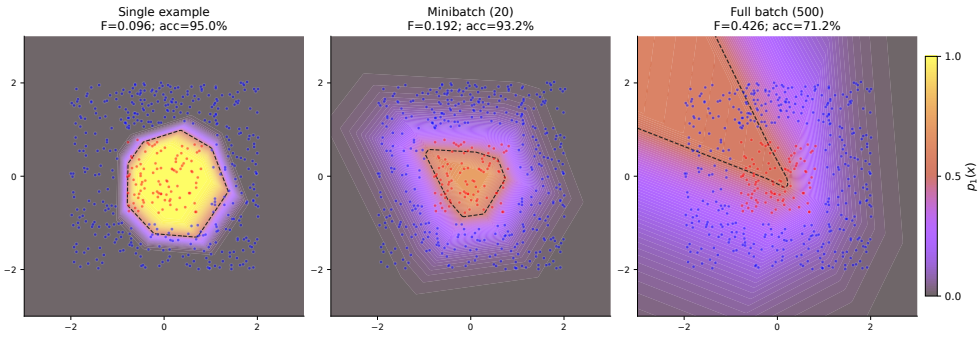
Batch size changes how our actual network learns



After 1 epoch: each method processed 500 examples; updates: 500 / 25 / 1.

Same starting network (after our 7 updates), $\eta = 0.20$; dashed black boundary $p_1 = 0.5$.

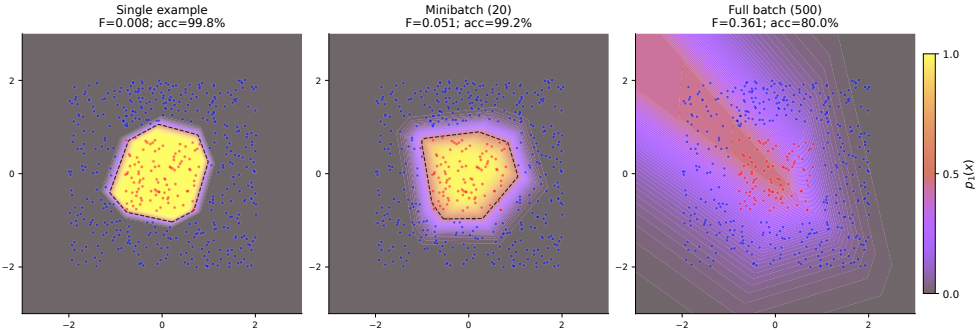
Batch size changes how our actual network learns



After 5 epochs: output maps already differ markedly despite equal example counts.

Same starting network (after our 7 updates), $\eta = 0.20$; dashed black boundary $p_1 = 0.5$.

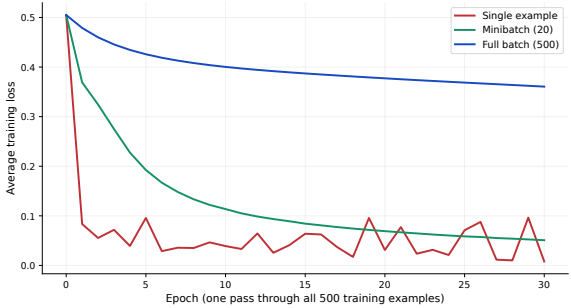
Batch size changes how our actual network learns



After 30 epochs: the update frequency matters greatly for this fixed η.

Same starting network (after our 7 updates), $\eta = 0.20$; dashed black boundary $p_1 = 0.5$.

Compare the average loss across epochs



At 30 epochs (15,000 example evaluations):

Method	F	Acc.
$B = 1$	0.0080	99.8%
$B = 20$	0.0509	99.2%
$B = 500$	0.3606	80.0%

Same learning rate, *not* the same number of updates. More frequent updates are important here, but not a universal ranking.

Why can the $B = 1$ method improve the average loss using only one example at a time?

Why should one example help? An unbiased gradient estimate

At a *fixed* parameter vector θ, choose an index $I \sim \text{Uniform}\{1, \dots, N\}$ and set $g_I(\theta) = \nabla \ell_I(\theta)$. Then

$$\mathbb{E}_I[g_I(\theta)] = \sum_{i=1}^{N} \underbrace{\Pr(I = i)}_{1/N} \nabla \ell_i(\theta) =$$

$$= \frac{1}{N} \sum_{i=1}^{N} \nabla \ell_i(\theta) = \boxed{\nabla F(\theta)}.$$

Unbiased means the average of hypothetical independently sampled gradients is the exact gradient. It does *not* mean any one step is correct, or that F decreases on every step.

For a sequence of updates, the standard theorem assumes $\mathbb{E}[g_t \mid \theta_t] = \nabla F(\theta_t)$ (*conditional* unbiasedness). Independent uniform sampling with replacement satisfies it. Our plotted shuffled-epoch experiment samples without replacement: it is a related practical algorithm, but not literally covered by this simple identity conditional on the evolving θ_t.

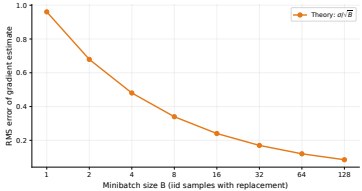
Why use minibatches? Variance falls as $1/B$

Draw B independent indices uniformly *with replacement*:

$$g_B = \frac{1}{B}\sum_{j=1}^{B} \nabla \ell_{I_j}(\theta).$$

At fixed θ, write $\sigma^2 = \mathbb{E}\|g_I - \nabla F\|^2$. Since independently sampled gradient errors have zero mean, their cross-products have zero expectation; only the B variance terms remain.

$$\boxed{\mathbb{E}\|g_B - \nabla F\|^2 = \frac{\sigma^2}{B}}.$$



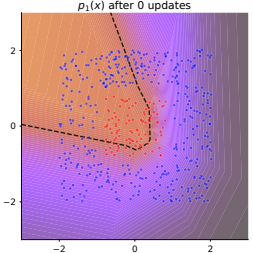
Computed from our network at θ_7: $\sigma^2 = 0.9230$ (squared Euclidean norm of the 22-dimensional gradient error).

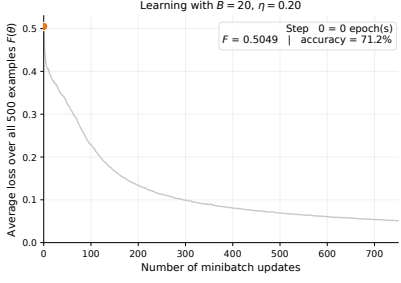
Averaging B independent gradients reduces their standard deviation by a factor of $\sqrt{B}$, but requires computing B example gradients per update.

Watch the network learn: 20 examples per update

Continue from our network after seven single-example updates. Now use **minibatch SGD**: at each step, average the gradients of $B = 20$ selected examples and update all 22 parameters:

$$g_t = \frac{1}{20} \sum_{i \in \mathcal{B}_t} \nabla \ell_i(\theta_t), \qquad \theta_{t+1} = \theta_t - 0.20\, g_t .$$



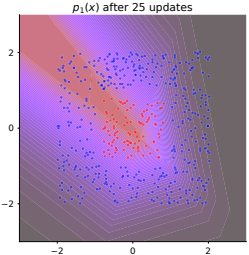


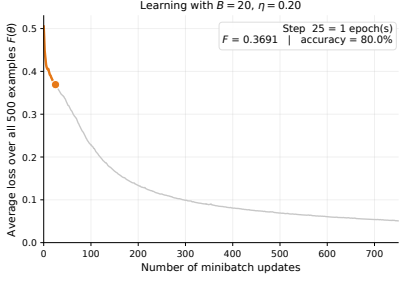
Curve: $F(\theta) = \frac{1}{500} \sum_{i=1}^{500} \ell_i(\theta)$ is *monitored*, while only the 20 minibatch examples determine the update. One epoch = 25 updates.

Watch the network learn: 20 examples per update

Continue from our network after seven single-example updates. Now use **minibatch SGD**: at each step, average the gradients of $B = 20$ selected examples and update all 22 parameters:

$$g_t = \frac{1}{20} \sum_{i \in \mathcal{B}_t} \nabla \ell_i(\theta_t), \qquad \theta_{t+1} = \theta_t - 0.20\, g_t .$$



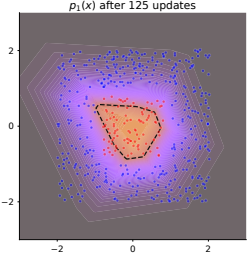


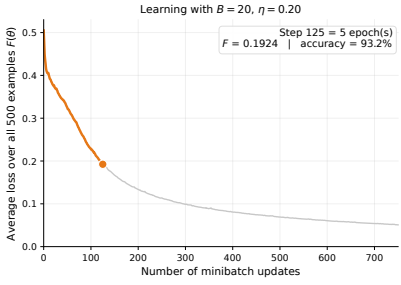
Curve: $F(\theta) = \frac{1}{500} \sum_{i=1}^{500} \ell_i(\theta)$ is *monitored*, while only the 20 minibatch examples determine the update. One epoch = 25 updates.

Watch the network learn: 20 examples per update

Continue from our network after seven single-example updates. Now use **minibatch SGD**: at each step, average the gradients of $B = 20$ selected examples and update all 22 parameters:

$$g_t = \frac{1}{20} \sum_{i \in \mathcal{B}_t} \nabla \ell_i(\theta_t), \qquad \theta_{t+1} = \theta_t - 0.20\, g_t .$$



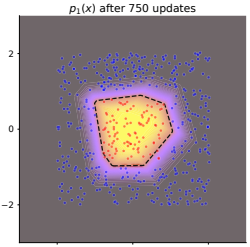


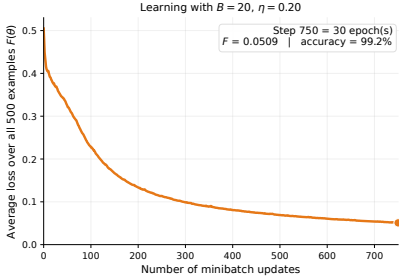
Curve: $F(\theta) = \frac{1}{500} \sum_{i=1}^{500} \ell_i(\theta)$ is *monitored*, while only the 20 minibatch examples determine the update. One epoch = 25 updates.

Watch the network learn: 20 examples per update

Continue from our network after seven single-example updates. Now use **minibatch SGD**: at each step, average the gradients of $B = 20$ selected examples and update all 22 parameters:

$$g_t = \frac{1}{20} \sum_{i \in \mathcal{B}_t} \nabla \ell_i(\theta_t), \qquad \theta_{t+1} = \theta_t - 0.20\, g_t .$$

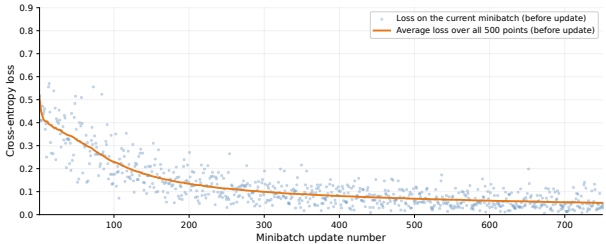




Curve: $F(\theta) = \frac{1}{500} \sum_{i=1}^{500} \ell_i(\theta)$ is *monitored*, while only the 20 minibatch examples determine the update. One epoch = 25 updates.

Two loss measurements: the minibatch and the whole dataset

The update uses the current minibatch's **mean loss**, $\ell_{\mathcal{B}}(\theta) = \frac{1}{20}\sum_{i\in\mathcal{B}} \ell_i(\theta)$. For diagnosis we can also compute $F(\theta)$, the mean over *all* 500 points.



Blue dots: loss on the selected 20 points *before* each update. It fluctuates because successive minibatches contain different observations.

Orange curve: mean loss across all 500 points at those same weights. It is more stable, but is not needed to compute an SGD update.

Learning rate: how far do the weights move?

At each minibatch step, the gradient g_t suggests how to change the 22-dimensional parameter vector θ:

$$\boxed{\theta_{t+1} = \theta_t - \eta g_t}, \qquad \eta > 0 \text{ is the } \textbf{learning rate}.$$

The *direction* comes from $-g_t$; the learning rate multiplies its *length*. For that update, $\|\theta_{t+1} - \theta_t\| = \eta\|g_t\|$.

Our network: For the first selected minibatch of 20 points, $\|g_0\| \approx 0.3800$. With *the same starting weights and minibatch*:

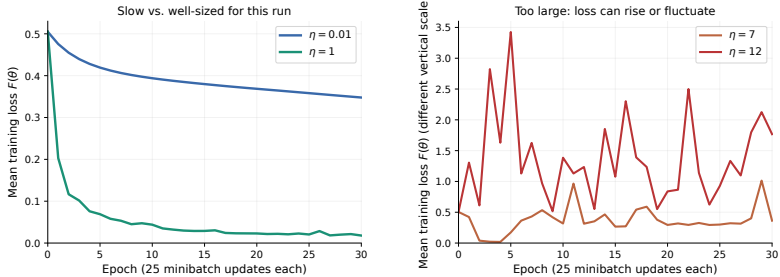
	Small step	Moderate step	Very large step
η	0.01	1	12
$\|\Delta\theta\| = \eta\|g_0\|$	0.0038	0.3800	4.5601

After the *first* update, each run reaches different weights, so subsequent gradients and trajectories need not remain proportional.

Note: here η values are higher than usually assumed (> 0.1, even > 1). This results from simplicity of dataset/examples construction.

Same network, same minibatches, four learning rates

All four runs begin at the *same* θ_7, visit the *same* minibatches in the same shuffled order, and use $B = 20$ for 30 epochs (750 parameter updates). **Only η changes.**

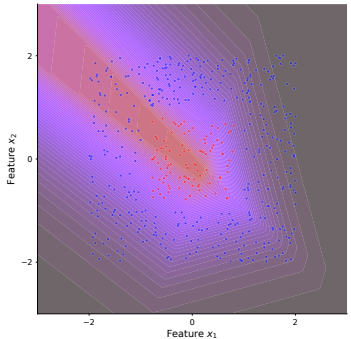


Both panels plot the same quantity, the **average training loss** $F(\theta)$, measured after each epoch. **The vertical scales differ.**

Here: $\eta = 0.01$ moves slowly; $\eta = 1$ reduces loss steadily; $\eta = 7$ is unstable after an initially rapid decrease; $\eta = 12$ often makes the loss *higher* than at the start.

What do those learning rates produce?

After 30 epochs, from the same starting weights and the same 750 minibatches:



Small $\eta = 0.01$

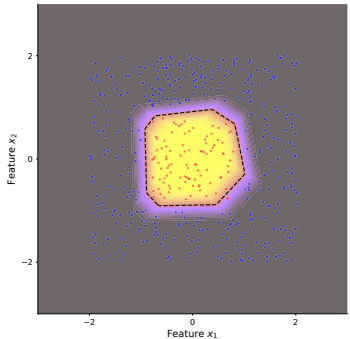
$F = 0.3478$; training accuracy 80.0%.

The network has changed, but slowly. It still predicts mostly class 0; 80% accuracy is possible simply by always predicting that majority class.

Blue/red dots are the *training examples*; the heatmap shows $p_1(x)$ with the same fixed $[0, 1]$ color scale on every overlay. Dashed boundary: $p_1 = 0.5$ when present.

What do those learning rates produce?

After 30 epochs, from the same starting weights and the same 750 minibatches:



Moderate $\eta = 1$

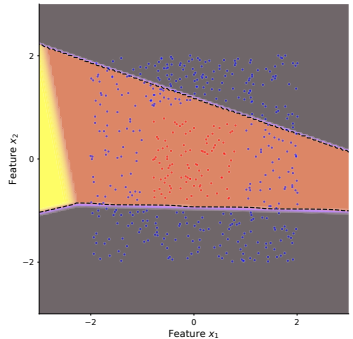
$F = 0.0180$; training accuracy 99.6%.

The network learned a narrow central region assigned to class 1. This rate worked well for this initialization and training budget.

Blue/red dots are the *training examples*; the heatmap shows $p_1(x)$ with the same fixed $[0, 1]$ color scale on every overlay. Dashed boundary: $p_1 = 0.5$ when present.

What do those learning rates produce?

After 30 epochs, from the same starting weights and the same 750 minibatches:



Large $\eta = 7$

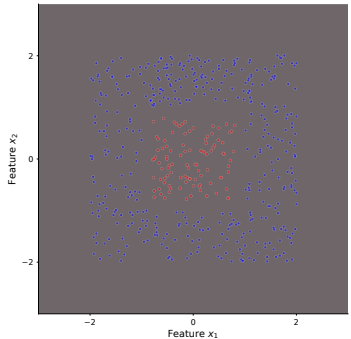
$F = 0.3630$; training accuracy 71.6%.

The loss fell extremely low at an earlier update, but later large steps disrupted that solution. A high rate need not maintain progress.

Blue/red dots are the *training examples*; the heatmap shows $p_1(x)$ with the same fixed $[0, 1]$ color scale on every overlay. Dashed boundary: $p_1 = 0.5$ when present.

What do those learning rates produce?

After 30 epochs, from the same starting weights and the same 750 minibatches:



Very large $\eta = 12$

$F = 1.7670$; training accuracy 80.0%.

At 30 epochs the loss is *worse than the initial* $F = 0.5049$. Large updates can overshoot and cause unstable training.

Blue/red dots are the *training examples*; the heatmap shows $p_1(x)$ with the same fixed $[0, 1]$ color scale on every overlay. Dashed boundary: $p_1 = 0.5$ when present.

How should we interpret a learning-rate curve?

The learning rate controls *how far we move*, not how many parameters are updated or how many examples are used in a minibatch.

Too small	Loss falls slowly; more updates may be needed. A slow start is not proof that learning has failed.
Useful range	Loss falls relatively fast without persistent violent oscillation. The best range depends on the network, data, and optimizer.
Too large	Loss may oscillate, repeatedly undo progress, or settle at a noisy error level rather than decrease further.
Far too large	Updates may overshoot badly: loss can rise, become unstable, or even become non-finite (`NaN`/`Inf`).

Important: A flat curve does *not* prove that we have found a minimum: small steps, noisy updates, inactive ReLUs, or insufficient training may all look flat. These four learning rates are examples, not universal thresholds.

Diagnosing training loss

Figure 1. Gradient descent in class

Diagnosing training loss

Figure 1. Gradient descent in class

Figure 2. Gradient descent in life

Diagnosing training loss

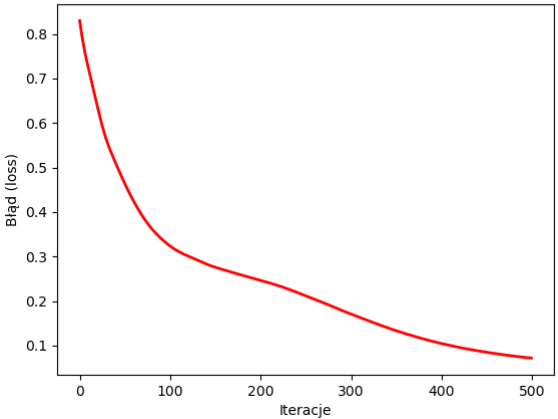


Figure 1. A nice, well behaving loss.

Diagnosing training loss

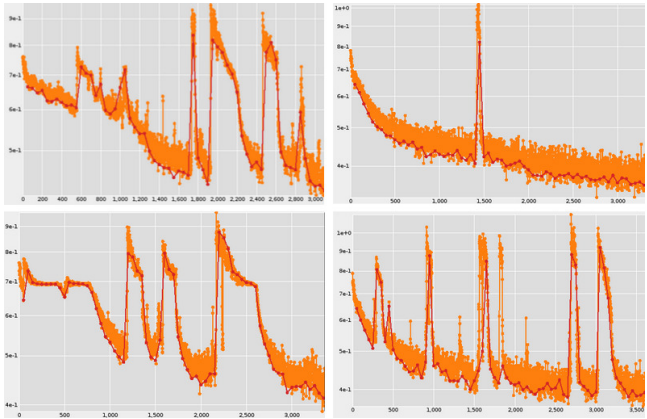


Figure 1. *Taming Spatial Transformer Networks, contributed by Diogo. For the record, it's not supposed to look like that.* (From `https://lossfunctions.tumblr.com/`)

Diagnosing training loss

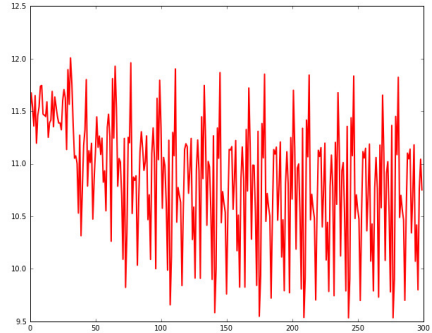


Figure 1. *A heart rate or a loss function? :) This one of a custom implementation of an RNN, graciously contributed by Ray Zhang.* (From `https://lossfunctions.tumblr.com/`)

Diagnosing training loss

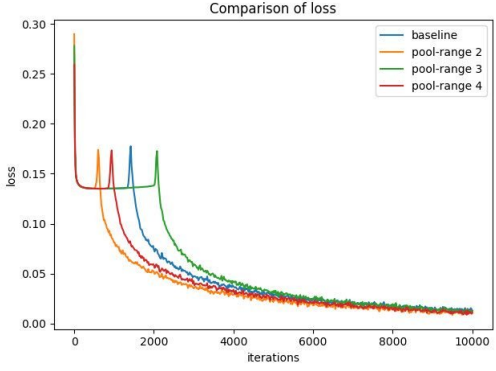
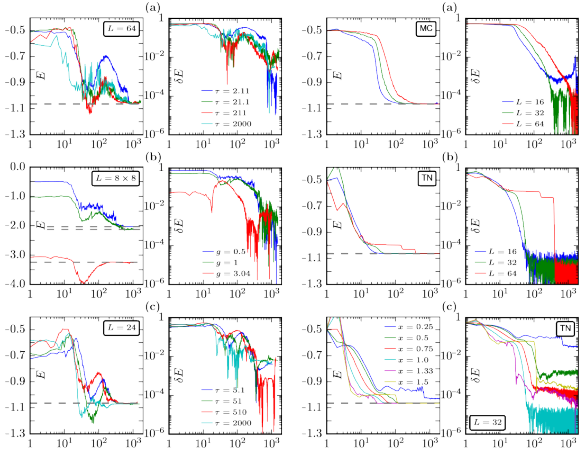


Figure 1. *A highly amusing specimen from @_karfly. Truly baffles the mind.* (From `https://lossfunctions.tumblr.com/`)

Diagnosing training loss



B. Gardas et al, Quantum neural networks to simulate many-body quantum systems, Phys. Rev. B 98, 184304 (2018)

Why move opposite to the gradient?

We want to minimize a loss function $f(\theta)$, where θ contains all trainable parameters.

1. The gradient describes how the loss changes.
For a small change Δ of the parameters:

$$\boxed{f(\theta + \Delta) \approx f(\theta) + \nabla f(\theta)^T \Delta}$$

Here $\nabla f(\theta)$ is the gradient (vector of partial derivatives), and the second term estimates the change in loss.

2. Choose a step opposite to the gradient.
Let $g = \nabla f(\theta)$ and $\Delta = -\eta g$, where $\eta > 0$ is the learning rate. Then:

$$\begin{aligned} f(\theta - \eta g) &\approx f(\theta) + g^T(-\eta g) \\ &= f(\theta) - \eta \underbrace{\|g\|^2}_{\Sigma_j g_j^2 \geq 0}. \end{aligned}$$

Thus, moving opposite to the gradient is expected to **decrease the loss**. In fact, $-g$ gives the steepest local decrease among directions of equal length (Euclidean norm).

3. But how large a step can we take?
The approximation is valid for sufficiently small steps. A large learning rate may actually *increase* the loss, because the gradient changes as we move. *Next question: under what mathematical conditions can we guarantee that a gradient step decreases the loss?*

Why does gradient descent not always reduce the loss?

Consider a one-parameter loss: the gradient gives a direction; the learning rate sets the step length.

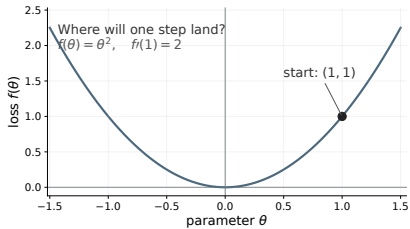
$$f(\theta) = \theta^2, \quad f'(\theta) = 2\theta, \quad \theta_0 = 1,\ f(\theta_0) = 1.$$

One gradient step, with learning rate $\eta > 0$, gives

$$\boxed{\theta_1 = \theta_0 - \eta f'(\theta_0) = 1 - 2\eta}, \qquad f(\theta_1) = (1 - 2\eta)^2.$$

Try four learning rates:

η	θ_1	$f(\theta_1)$



Question: The slope is positive, so we move left. How far?

Why does gradient descent not always reduce the loss?

Consider a one-parameter loss: the gradient gives a direction; the learning rate sets the step length.

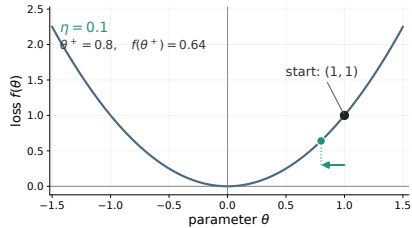
$$f(\theta) = \theta^2, \quad f'(\theta) = 2\theta, \quad \theta_0 = 1, \; f(\theta_0) = 1.$$

One gradient step, with learning rate $\eta > 0$, gives

$$\boxed{\theta_1 = \theta_0 - \eta f'(\theta_0) = 1 - 2\eta}, \qquad f(\theta_1) = (1 - 2\eta)^2.$$

Try four learning rates:

η	θ_1	$f(\theta_1)$
0.1	0.8	0.64



Small step: the loss decreases, but we are still far from the minimum.

Why does gradient descent not always reduce the loss?

Consider a one-parameter loss: the gradient gives a direction; the learning rate sets the step length.

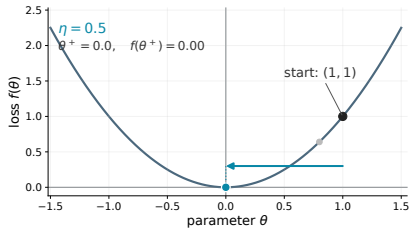
$$f(\theta) = \theta^2, \quad f'(\theta) = 2\theta, \quad \theta_0 = 1, \; f(\theta_0) = 1.$$

One gradient step, with learning rate $\eta > 0$, gives

$$\boxed{\theta_1 = \theta_0 - \eta f'(\theta_0) = 1 - 2\eta}, \qquad f(\theta_1) = (1 - 2\eta)^2.$$

Try four learning rates:

η	θ_1	$f(\theta_1)$
0.1	0.8	0.64
0.5	0	0



Here: the chosen step reaches the minimum exactly.

Why does gradient descent not always reduce the loss?

Consider a one-parameter loss: the gradient gives a direction; the learning rate sets the step length.

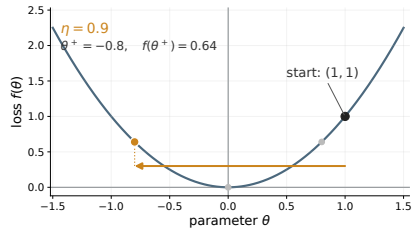
$$f(\theta) = \theta^2, \quad f'(\theta) = 2\theta, \quad \theta_0 = 1,\ f(\theta_0) = 1.$$

One gradient step, with learning rate $\eta > 0$, gives

$$\boxed{\theta_1 = \theta_0 - \eta f'(\theta_0) = 1 - 2\eta}, \qquad f(\theta_1) = (1 - 2\eta)^2.$$

Try four learning rates:

η	θ_1	$f(\theta_1)$
0.1	0.8	0.64
0.5	0	0
0.9	−0.8	0.64



Overshoot: we cross the minimum, but the loss still decreases.

Why does gradient descent not always reduce the loss?

Consider a one-parameter loss: the gradient gives a direction; the learning rate sets the step length.

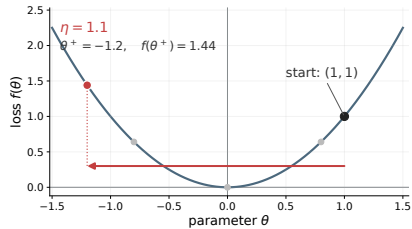
$$f(\theta) = \theta^2, \quad f'(\theta) = 2\theta, \quad \theta_0 = 1, \; f(\theta_0) = 1.$$

One gradient step, with learning rate $\eta > 0$, gives

$$\boxed{\theta_1 = \theta_0 - \eta f'(\theta_0) = 1 - 2\eta}, \qquad f(\theta_1) = (1 - 2\eta)^2.$$

Try four learning rates:

η	θ_1	$f(\theta_1)$
0.1	0.8	0.64
0.5	0	0
0.9	−0.8	0.64
1.1	−1.2	1.44



Too far: the direction is right, but the loss increases above 1.

Can we guarantee that a gradient step decreases the loss?

1. For our quadratic example, the answer is exact.
Recall $f(\theta) = \theta^2$ and $f'(\theta) = 2\theta$. A gradient step changes *any* starting value θ into

$$\theta^+ = \theta - \eta f'(\theta) = (1 - 2\eta)\theta.$$

Therefore the new loss is

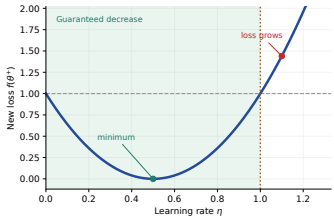
$$\boxed{f(\theta^+) = (1 - 2\eta)^2 \, f(\theta).}$$

The loss decreases whenever $(1 - 2\eta)^2 < 1$, i.e.

$$\boxed{0 < \eta < 1.}$$

This is a guarantee for **every** $\theta \neq 0$, not just the point from the previous slide.

At $\eta = 1$ the loss is unchanged; for $\eta > 1$ it grows. The limit 1 belongs to *this particular function*.



Can we guarantee that a gradient step decreases the loss?

2. For a general loss, we need an assumption.
Different loss functions bend differently. We assume that the gradient cannot change *too quickly*. This is called M**-smoothness**:

$$\boxed{\|\nabla f(v) - \nabla f(u)\| \leq M\|v - u\|.}$$

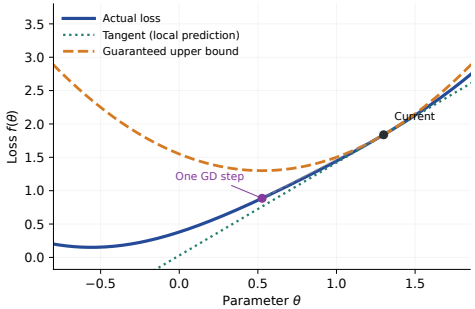
$M > 0$ limits how rapidly the gradient (slope) changes as the parameters move.

It gives the **descent lemma**:

$$f(v) \leq f(u) + \nabla f(u)^T(v - u) + \frac{M}{2}\|v - u\|^2.$$

The first two terms are the *tangent prediction*; the last term allows for curvature.

Unlike a local approximation, this is a **guaranteed upper bound**.



Can we guarantee that a gradient step decreases the loss?

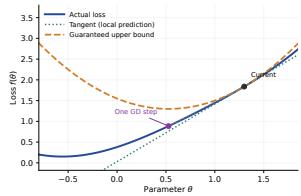
3. The bound gives a safe learning-rate range.
Put $g = \nabla f(u)$ and use the exact gradient step $u^+ = u - \eta g$. The descent lemma gives

$$\begin{aligned} f(u^+) &\le f(u) - \eta\|g\|^2 + \frac{M\eta^2}{2}\|g\|^2 \\ &= f(u) - \eta\left(1 - \frac{M\eta}{2}\right)\|g\|^2. \end{aligned}$$

For $0 < \eta < 2/M$, the correction is nonnegative:

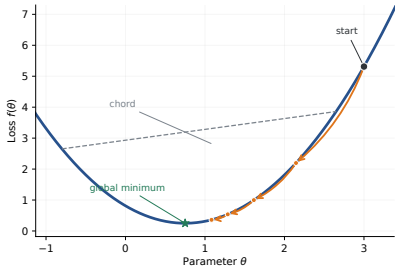
$$\boxed{0 < \eta < \frac{2}{M} \quad \Rightarrow \quad f(u^+) \le f(u).}$$

This is how we move from "*gradient descent should work for a sufficiently small step*" to "*under these assumptions, we can prove this step will not increase the loss.*"



Decreasing the loss is not the same as finding the minimum

A gradient step may lower loss. But does repetition find the best solution?



1. Convex function: no misleading valleys.

$$f(\theta) = (\theta - 0.75)^2 + 0.25.$$

Convex means the graph lies *below any chord* joining two of its points (dashed line):

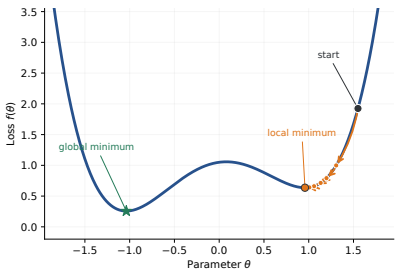
$$f((1-t)u + tv) \\ \leq (1-t)f(u) + tf(v), \quad 0 \leq t \leq 1.$$

The orange steps move toward the lowest point.

Key fact: For differentiable convex f, $f'(\theta) = 0$ implies a *global minimum*.

Decreasing the loss is not the same as finding the minimum

A gradient step may lower loss. But does repetition find the best solution?



2. Nonconvex function: several valleys.

$$f(\theta) = 0.6(\theta^2 - 1)^2 + 0.19\theta + 0.45.$$

Local minimum: no *nearby* point has lower loss.

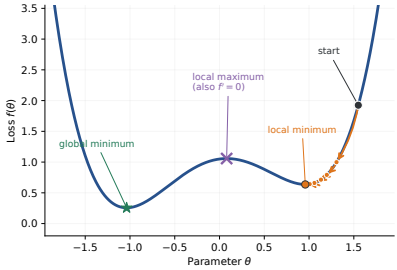
Global minimum: no point *anywhere* has lower loss.

Starting on the right, every orange GD step lowers the loss, but the updates approach the *higher* right-hand valley.

The green star marks the lower global minimum: our updates do not reach it.

Decreasing the loss is not the same as finding the minimum

A gradient step may lower loss. But does repetition find the best solution?



3. What does zero gradient tell us?

$$f'(\theta) = 0$$

This is a **stationary point**: an exact GD update cannot move from it.

All three marked points have $f'(\theta) = 0$:

- green star: global minimum;
- orange circle: local minimum;
- purple cross: local maximum.

With multiple parameters, *saddles* are also possible: downhill in some directions, uphill in others.

Our neural network: nonconvex loss in 22 parameters.

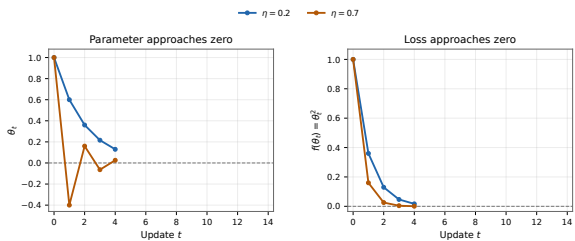
Does gradient descent converge?

1. One safe step is useful. What happens after many steps?
For $f(\theta) = \theta^2$ with $f'(\theta) = 2\theta$ and $\theta_0 = 1$, each update uses the *new* parameter value:

$$\theta_{t+1} = \theta_t - \eta f'(\theta_t) = (1 - 2\eta)\theta_t.$$

First four updates:

Update t	$\eta = 0.2$	$\eta = 0.7$
0	1	1
1	0.600	−0.400
2	0.360	0.160
3	0.216	−0.064
4	0.130	0.026



With $\eta = 0.7$, the parameter *crosses zero* repeatedly, yet its loss keeps decreasing.

Does gradient descent converge?

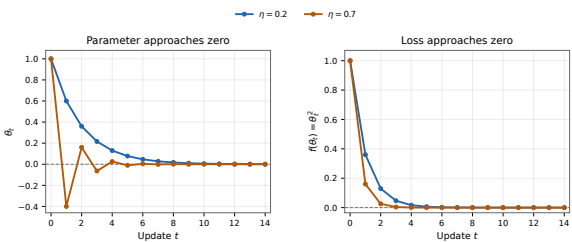
2. For this example, we can actually prove convergence.
Each update multiplies the parameter by the same number $1 - 2\eta$. After T updates:

$$\theta_T = (1 - 2\eta)^T \theta_0, \qquad f(\theta_T) = (1 - 2\eta)^{2T} f(\theta_0).$$

If $0 < \eta < 1$, then $q = |1 - 2\eta| < 1$, and repeated multiplication gives $q^T \to 0$.

$$\theta_T \to 0, \qquad f(\theta_T) \to 0.$$

Convergence: the parameter approaches $\theta = 0$ (the global minimum), even if no finite step lands there.



Does gradient descent converge?

3. What can we conclude for more general problems?

Smooth and convex loss

Assume the gradient changes at a bounded rate (M-smoothness), and a global minimum exists. With exact gradients and a suitable fixed learning rate:

$$\boxed{f(\theta_T) \longrightarrow f_{\min}.}$$

In words: the *loss value* approaches the best possible value.

Smooth, but nonconvex loss

Assume M-smoothness and that the loss has a lower bound. With exact gradients and a suitable fixed learning rate:

$$\boxed{\|\nabla f(\theta_T)\| \longrightarrow 0.}$$

In words: gradient descent can approach points where the gradient is nearly zero, but these points are not necessarily global minima.

Important limitations. Our ReLU network has non-differentiable boundaries, so these smooth-function theorems do not directly apply. Also, minibatch SGD uses *noisy* gradients: individual steps need not reduce the full-dataset loss.

Next question: What changes when the gradient is stochastic?

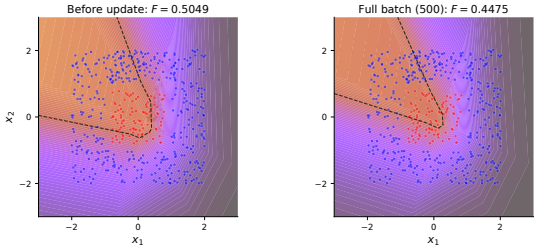
What changes when we use SGD?

Our goal is unchanged: minimize average cross-entropy on 500 examples,

$$F(\theta) = \frac{1}{500} \sum_{i=1}^{500} \ell(\theta; x_i, y_i).$$

Full-batch gradient descent: calculate the gradient using *all 500 examples*.

$$g = \nabla F(\theta), \qquad \theta^+ = \theta - 0.5g.$$



Measured: $F(\theta_7) = 0.5049 \longrightarrow F(\theta^+) = 0.4475.$

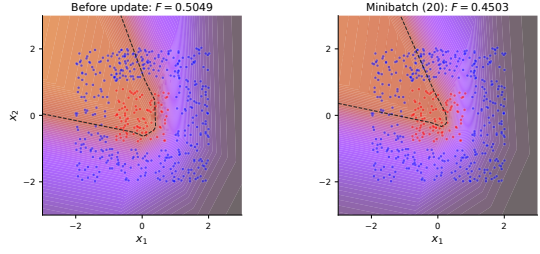
What changes when we use SGD?

Our goal is unchanged: minimize average cross-entropy on 500 examples,

$$F(\theta) = \frac{1}{500} \sum_{i=1}^{500} \ell(\theta; x_i, y_i).$$

Minibatch SGD: randomly choose 20 examples and average *their* gradients.

$$g_B = \frac{1}{20} \sum_{i \in \mathcal{B}} \nabla \ell_i(\theta), \qquad \theta^+ = \theta - 0.5 g_B.$$



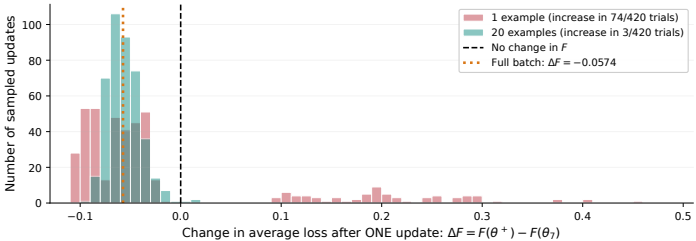
Measured from the same start: $F : 0.5049 \longrightarrow 0.4503$.

What changes when we use SGD?

Our goal is unchanged: minimize average cross-entropy on 500 examples,

$$F(\theta) = \frac{1}{500} \sum_{i=1}^{500} \ell(\theta; x_i, y_i).$$

One update can help or hurt. Keep θ_7 and $\eta = 0.5$ fixed. Draw 420 minibatches for each size, always restarting from the same weights.

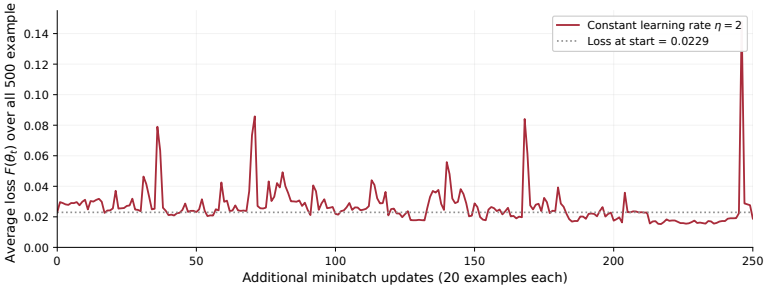


Result: 74/420 one-example updates raised F, versus 3/420 for $B = 20$. **Unbiased does not mean every step decreases the loss.**

What changes when we repeat minibatch updates?

Continue our **same 2–4–2 network** after 20 epochs of minibatch training ($\eta = 1$). The average loss over 500 examples is already $F = 0.0229$. Each new step uses a randomly sampled minibatch of 20 examples.

Keep learning rate fixed, $\eta = 2$: updates often improve F, but some push it sharply upward. The full-data loss is measured *after every update*.

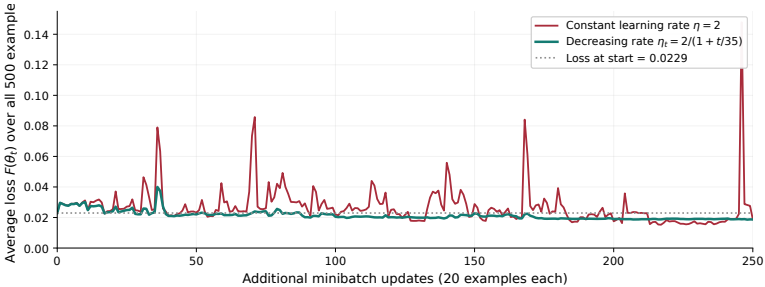


Why? The minibatch gradient depends on which 20 observations were drawn; it can differ from the full gradient even when the overall loss is already small.

What changes when we repeat minibatch updates?

Continue our **same 2–4–2 network** after 20 epochs of minibatch training ($\eta = 1$). The average loss over 500 examples is already $F = 0.0229$. Each new step uses a randomly sampled minibatch of 20 examples.

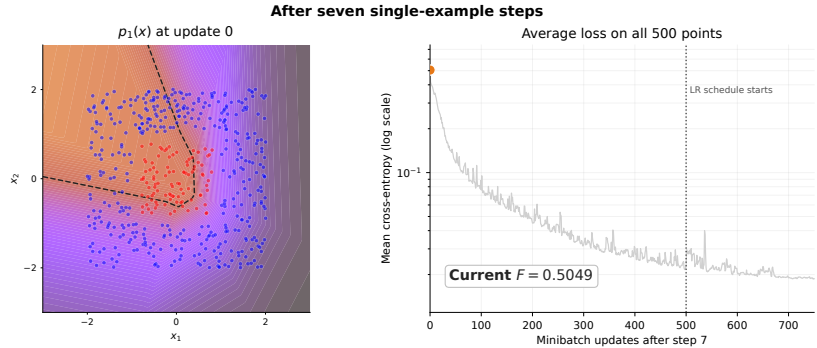
Reduce the learning rate as training continues: $\eta_t = 2/(1 + t/35)$ (green), versus constant $\eta = 2$ (red). Both runs start at the same weights and use the *same sequence of sampled minibatches*.



Here, smaller later steps limit loss spikes. They do *not* make every step improve the loss, nor prove convergence to a global minimum.

Finishing training: watch the classifier emerge

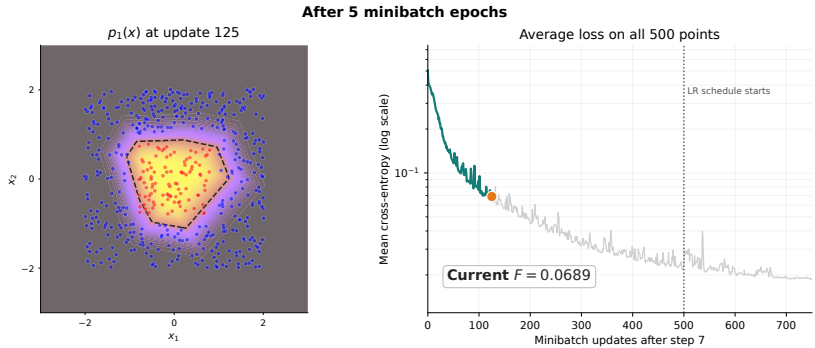
Continue our **same 2–4–2 network after step 7**, with 20 examples per update: 20 epochs at $\eta = 1$ (500 updates), followed by 250 further updates with a decreasing rate $\eta_t = 2/(1 + t/35)$ ($t = 0, \ldots, 249$).



Left: probability of class 1, $p_1(x)$, at successive checkpoints; black dashed boundary: $p_1 = 0.5$. **Right:** average cross-entropy over all 500 training examples, plotted on a *logarithmic vertical scale*. The 20 sampled examples—not the full dataset—determine each update.

Finishing training: watch the classifier emerge

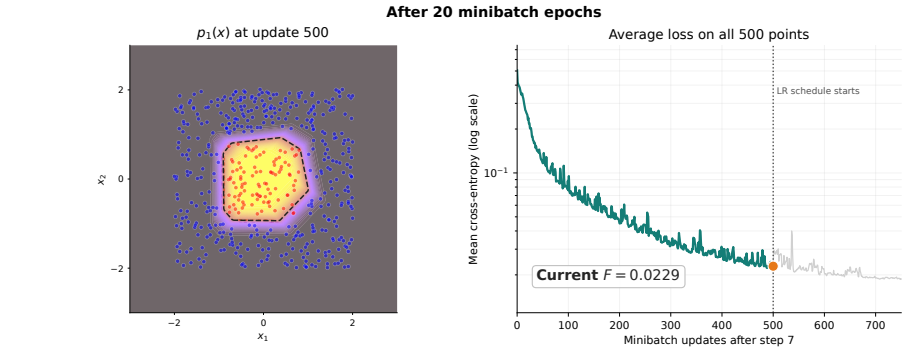
Continue our **same 2–4–2 network after step 7**, with 20 examples per update: 20 epochs at $\eta = 1$ (500 updates), followed by 250 further updates with a decreasing rate $\eta_t = 2/(1 + t/35)$ $(t = 0, \ldots, 249)$.



Left: probability of class 1, $p_1(x)$, at successive checkpoints; black dashed boundary: $p_1 = 0.5$. **Right:** average cross-entropy over all 500 training examples, plotted on a *logarithmic vertical scale*. The 20 sampled examples—not the full dataset—determine each update.

Finishing training: watch the classifier emerge

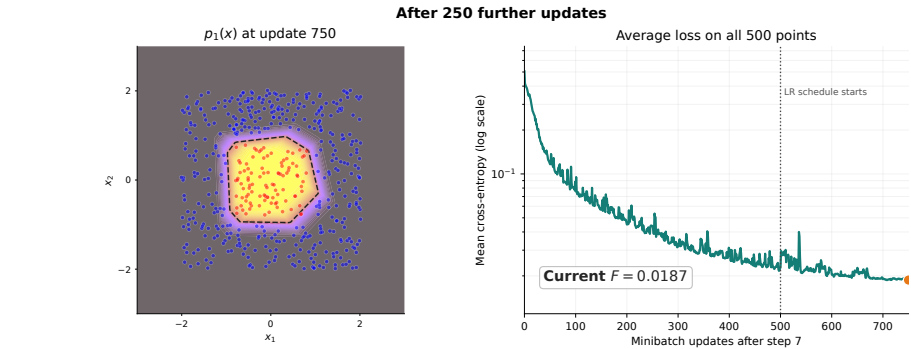
Continue our **same 2–4–2 network after step 7**, with 20 examples per update: 20 epochs at $\eta = 1$ (500 updates), followed by 250 further updates with a decreasing rate $\eta_t = 2/(1 + t/35)$ $(t = 0, \ldots, 249)$.



Left: probability of class 1, $p_1(x)$, at successive checkpoints; black dashed boundary: $p_1 = 0.5$. **Right:** average cross-entropy over all 500 training examples, plotted on a *logarithmic vertical scale*. The 20 sampled examples—not the full dataset—determine each update.

Finishing training: watch the classifier emerge

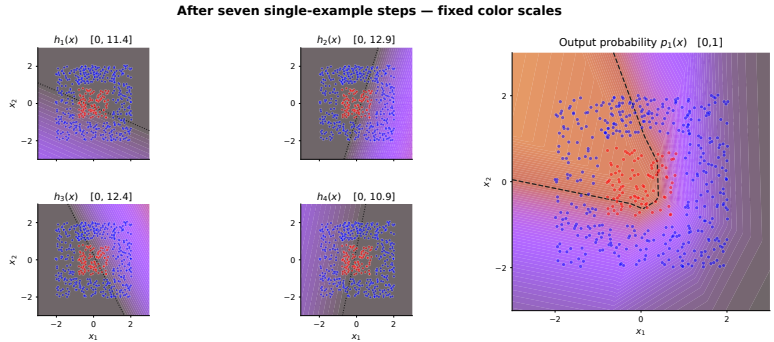
Continue our **same 2–4–2 network after step 7**, with 20 examples per update: 20 epochs at $\eta = 1$ (500 updates), followed by 250 further updates with a decreasing rate $\eta_t = 2/(1 + t/35)$ $(t = 0, \ldots, 249)$.



Left: probability of class 1, $p_1(x)$, at successive checkpoints; black dashed boundary: $p_1 = 0.5$. **Right:** average cross-entropy over all 500 training examples, plotted on a *logarithmic vertical scale*. The 20 sampled examples—not the full dataset—determine each update.

How did the four hidden neurons change?

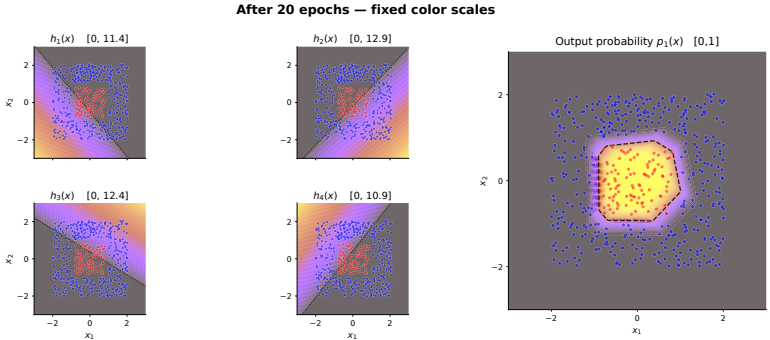
Each hidden neuron still computes $h_j(x) = \max(0, xw_j + b_j)$. Training changes its weights w_j and bias b_j—therefore its *entire activation map*, not just the value at a sampled observation.



Four small plots: hidden ReLU activations; dotted straight line: where a neuron switches from inactive to active. **Large plot:** output probability; dashed contour: the classifier's decision boundary. The color scales are *fixed across overlays* for each neuron.

How did the four hidden neurons change?

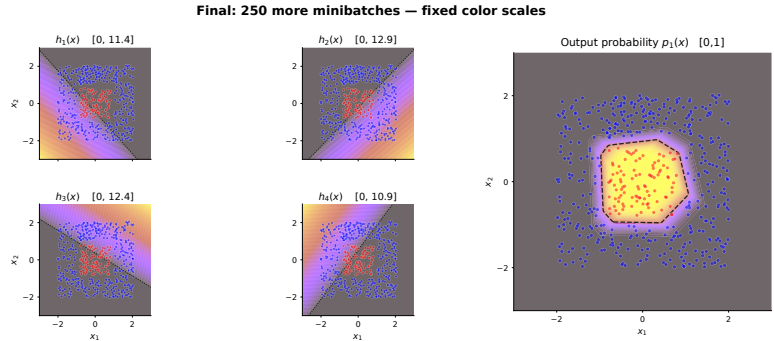
Each hidden neuron still computes $h_j(x) = \max(0, xw_j + b_j)$. Training changes its weights w_j and bias b_j—therefore its *entire activation map*, not just the value at a sampled observation.



Four small plots: hidden ReLU activations; dotted straight line: where a neuron switches from inactive to active. **Large plot:** output probability; dashed contour: the classifier's decision boundary. The color scales are *fixed across overlays* for each neuron.

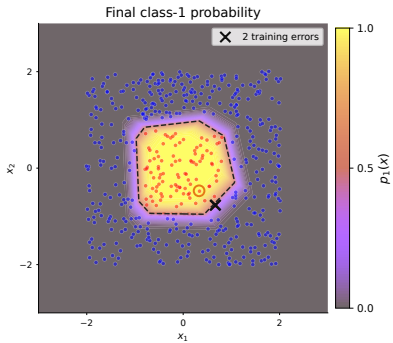
How did the four hidden neurons change?

Each hidden neuron still computes $h_j(x) = \max(0, xw_j + b_j)$. Training changes its weights w_j and bias b_j—therefore its *entire activation map*, not just the value at a sampled observation.



Four small plots: hidden ReLU activations; dotted straight line: where a neuron switches from inactive to active. **Large plot:** output probability; dashed contour: the classifier's decision boundary. The color scales are *fixed across overlays* for each neuron.

Final result: what has our network learned?



Average training loss:

$$F(\theta) = 0.0187$$

Training accuracy: $498/500 = 99.6\%$.
Prediction counts (rows = true class):

	Pred. 0	Pred. 1
True 0	400	0
True 1	2	98

Our original point $(0.33, -0.47)$: p_1 rose from 0.2769 at initialization to 0.9879.

Crucial qualification: these 500 points were *used for training*. A trivial majority-class model would get 80% here; 99.6% is a large improvement, but it is *not* a test of generalization.

What did this experiment establish?

What we demonstrated

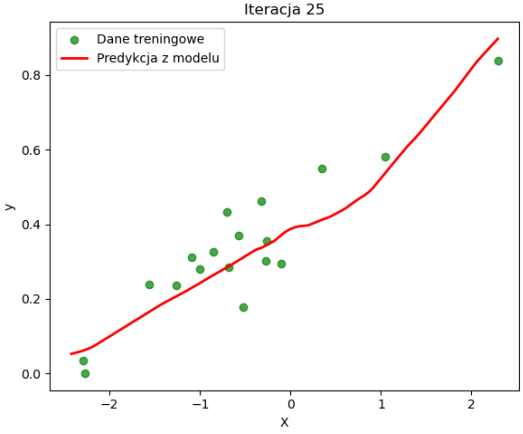
- **Forward pass:** inputs → ReLU hidden neurons → logits → softmax class probabilities.
- **Loss:** cross-entropy assigns a numerical penalty to a wrong probability prediction.
- **Backpropagation:** the chain rule gives gradients for all 22 trainable weights and biases.
- **Training:** many noisy minibatch updates can greatly reduce average training loss. Step size affects progress.

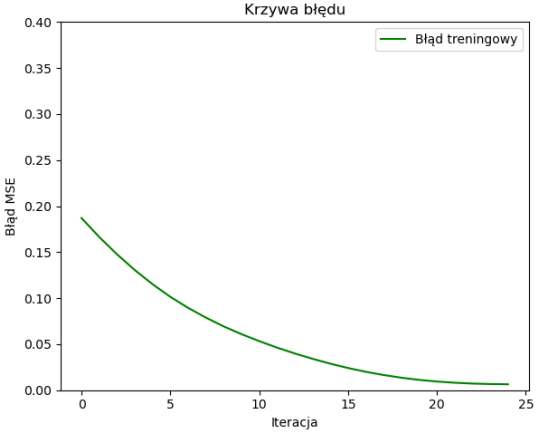
What we have not established

- **Generalization:** we have no held-out examples. 99.6% training accuracy is *not* test accuracy.
- **Best training practice:** we have not examined scaling, regularization (weight decay, dropout), momentum, or Adam.
- **Global optimality:** a low measured loss does not prove that a nonconvex network reached the best possible solution.

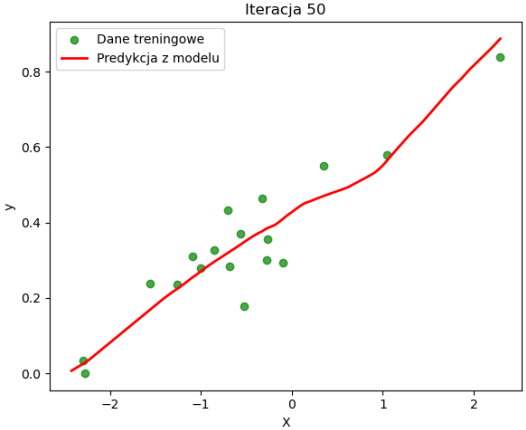
Next research question: Does this classifier work on new, unseen observations—and how would we measure that reliably?

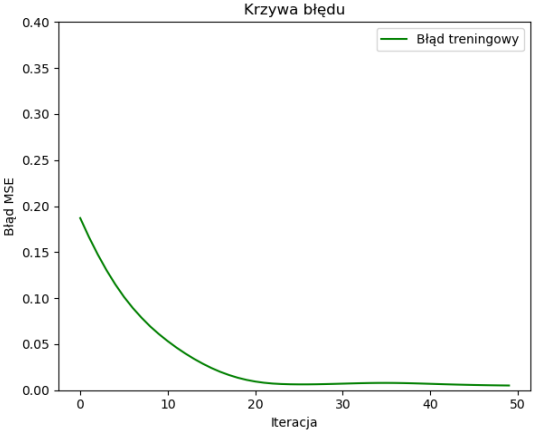
Generalization



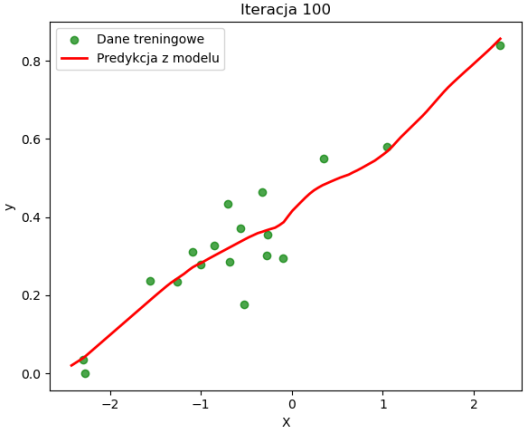


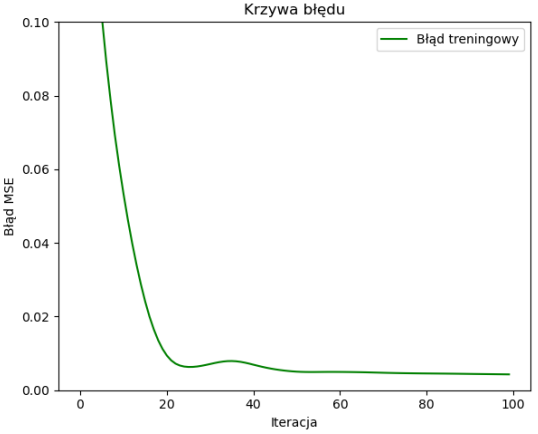
Generalization



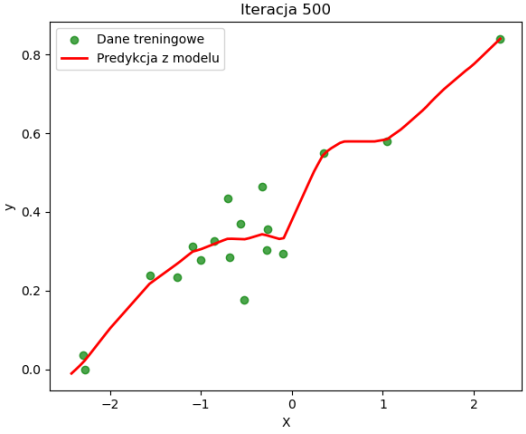


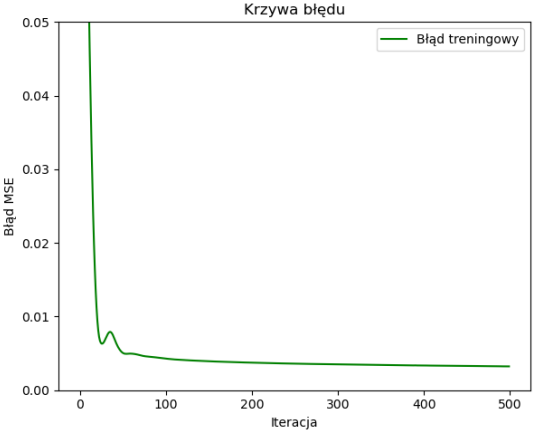
Generalization



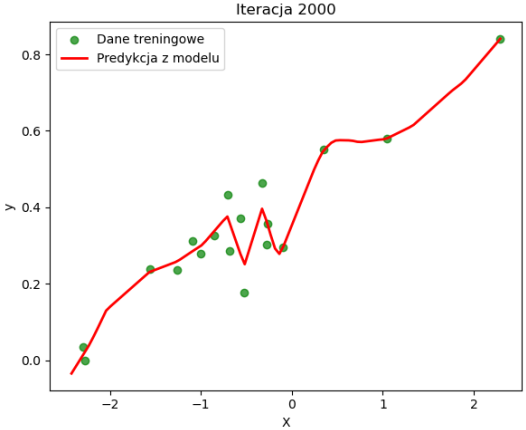


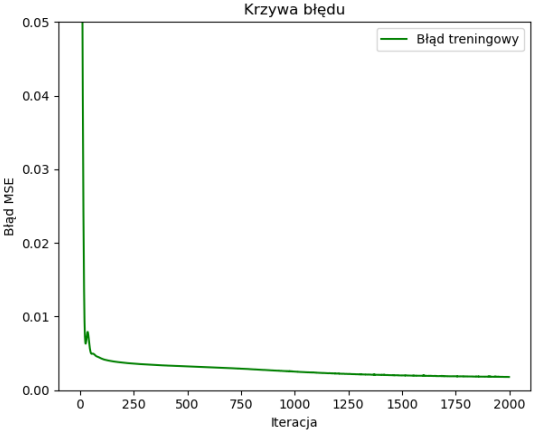
Generalization



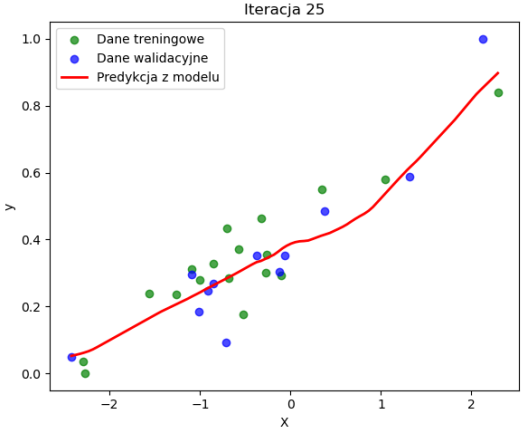


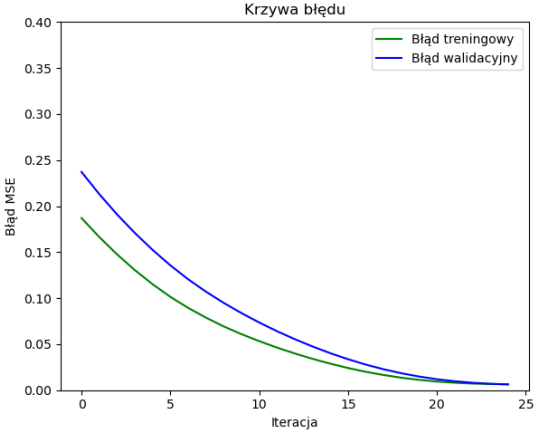
Generalization



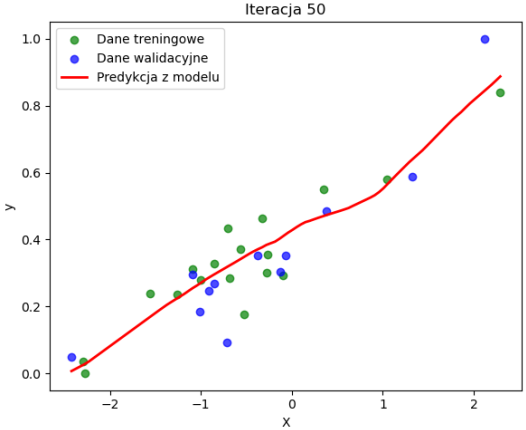


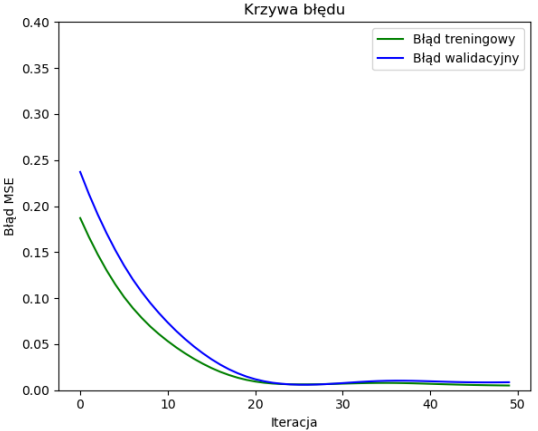
Generalization



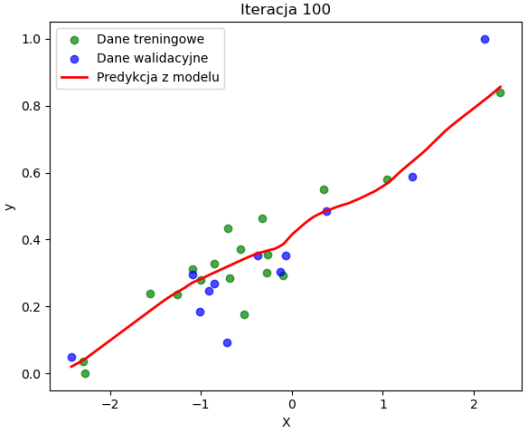


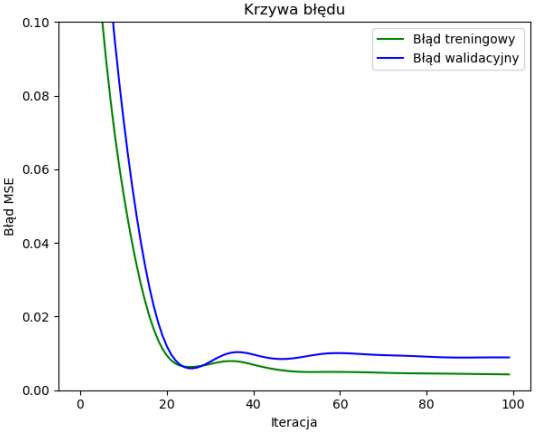
Generalization



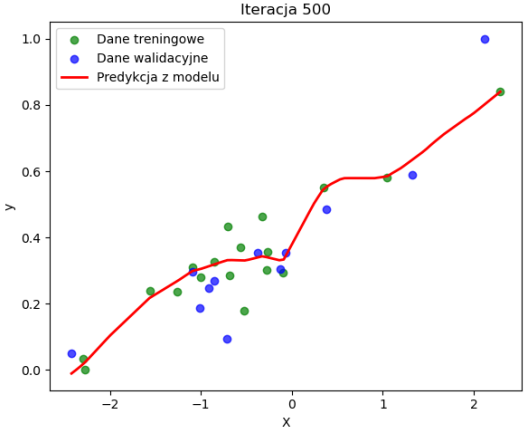


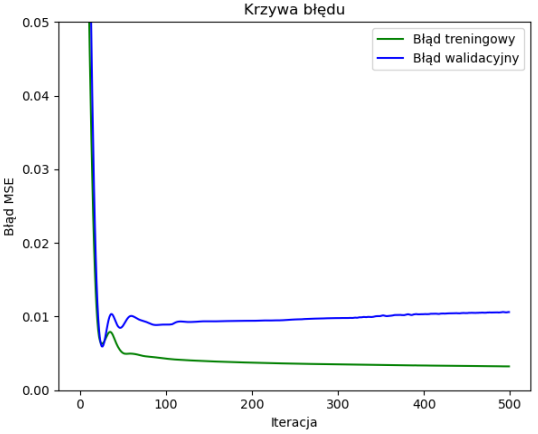
Generalization



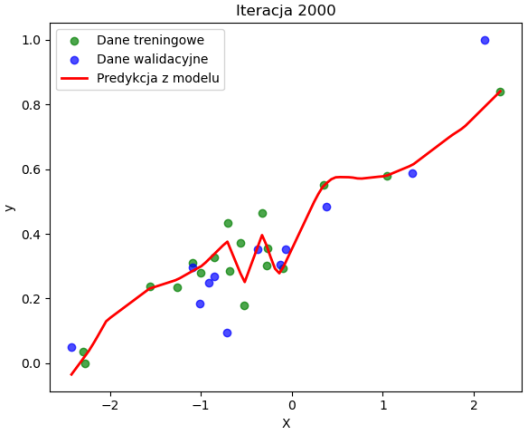


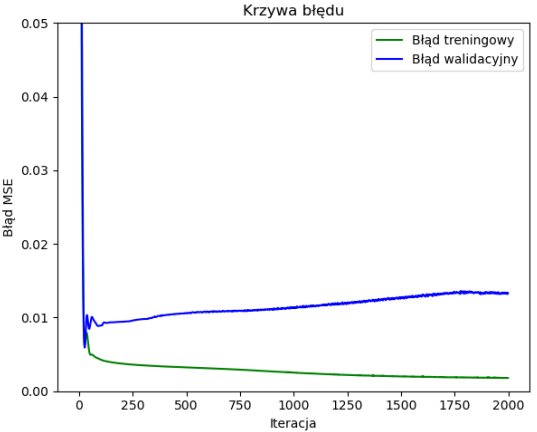
Generalization





Generalization





Beyond our 2–4–2 network

Our example used ReLU, softmax, cross-entropy and minibatch SGD.

How else can we train?

- **Input standardization:** subtract each feature's mean and divide by its standard deviation; comparable scales often make optimization easier.
- **Regularization:** L2 penalizes large weights; L1 encourages sparse weights; dropout randomly disables units during training.
- **Batch normalization:** normalizes intermediate activations using minibatch statistics; affects training dynamics and may also regularize.
- **Optimizers:** momentum uses accumulated gradient directions; Adam adapts updates per parameter; learning-rate schedules change the step size during training.

What else can a network do?

- **Other activations:** sigmoid, tanh, Leaky ReLU and GELU produce different nonlinear responses and gradients.
- **Different outputs and losses:** regression uses a numeric output (e.g., squared-error loss); one-of-K classification uses softmax and cross-entropy; multilabel classification uses independent sigmoids with binary cross-entropy.
- **Deeper networks:** layers combine features in stages, but backpropagated gradients may shrink (*vanish*) or grow (*explode*) across many layers.